CCL-2: Arithmetic and Finite Calculus with Complex Continued Logarithms

Anonymous

August 14, 2026

Abstract

CCL-1 constructs finite complex continued-logarithm words, ordinary raw-prefix parser predicates, shrinking projective cylinders, and progressive rational boxed names over the Gaussian rationals. This second paper develops the arithmetic layer on that representation. Its redundant output sectors overlap, so a whole rational enclosure can justify a next digit without deciding an unstable canonical boundary. A rational squared-error sidecar transports addition, subtraction, multiplication, and apartness-certified division. Projective unary padding handles exact results that cannot be recognized from opaque non-terminal inputs, and a persistent finite-request cursor returns every requested number of ordinary CCL bits.

The same interface computes exponential, positive-real logarithm, and complex logarithm in an explicit branch chart. All local formulae are finite rational Horner recurrences with displayed error bounds. Global range preparation is a bounded dyadic calculation followed by finitely many arithmetic operations. Operational equivalence proves the field laws, exponential addition, exponential–logarithm inverse laws on certified charts, and congruence under independent refinement schedules.

Finally, finite divided differences give limit-free differential laws, first- and higher-order jets, Taylor error bounds, coefficient-defined and progressive path integration, trigonometric compositions, projective rational-function evaluation, finite families of fractional powers, and an explicit coarse bit-complexity account. Every result is a terminating request over displayed finite data. No completed stream, completed scalar field, infinitesimal, limit, or function represented as an infinite graph is formed.

Introduction

CCL-1 constructs the representation consumed here. Its finite raw parser associates each ordinary bit prefix with an exact rational projective matrix and a finite predicate on admissible Gaussian-rational tails. A completed canonical block contracts the Hermitian projective quadrance

$$\chi^2\big([x_0 : x_1], [y_0 : y_1]\big) = \frac{\mathrm{N}(x_0y_1 - y_0x_1)}{\big(\mathrm{N}(x_0) + \mathrm{N}(x_1)\big)\big(\mathrm{N}(y_0) + \mathrm{N}(y_1)\big)}$$

of two compatible tails by at most one half. This is the Gaussian-norm, or Hermitian, extension of projective quadrance [1]. The boxed-name interface answers a positive rational accuracy request with a longer finite word, a nested rational box, a displayed rational witness, and proofs of enclosure and prefix persistence.

Those are the only representation-level facts assumed below. This paper adds the redundant-output and quantitative sidecar layer required for productive arithmetic. It first applies that layer to the shrinking zero/pole tracks supplied by the RFAP paper, then develops elementary arithmetic, exponential and logarithmic transforms, operational laws, limit-free calculus, integration, and finite root families. The canonical encoder and its half-open boundary conventions are never changed.

The same one-way boundary also couples the finite approximation-sharpness runs of RFAP-2 to ordinary CCL prefixes. One final finite run is reserved first; intermediate states then reveal longer initial segments of its window list and of one rational sample's terminating word. This gives a prefix-coherent finite tower without asserting an infinite source or compatibility between independently computed final runs.

As in CCL-1, every numerical object is finite rational data. A Gaussian rational is written $x + iy$ with squared norm $\mathrm{N}(x + iy) = x^2 + y^2$, and $\gamma = 1 + i$ has norm two. A "name" is a terminating request interface

returning a finite word and finite certificates, not an actual infinite stream or a completed point. The formal projective coordinate $\omega = [1 : 0]$ is only a pair of rational coordinates. All searches range over displayed finite lists or bounded natural-number intervals.

1 Wrapping finite FTA and RFAP box tracks

The CCL construction does not replace the winding and Sturm calculation of the companion finite FTA and RFAP theorem. It consumes its output. For a displayed polynomial, that theorem first performs finite squarefree preprocessing. Every linear factor supplies an exact Gaussian rational zero. Every higher factor supplies a degree-indexed family of pairwise separated winding-one tracks:

$$(B_{j,0}, B_{j,1}, \ldots), \qquad B_{j,n+1} \subseteq B_{j,n}, \qquad \Delta^2(B_{j,n}) \leq \varepsilon_n^2 \tag{1.1}$$

operationally, one requested finite refinement at a time. An exact non-zero terminal is passed to the finite CCL encoder of CCL-1; zero retains its distinguished finite tag. A higher track itself supplies a progressive squared-error process: at a finite state its candidate predicate is membership in the displayed box, its ball is based at the rational box anchor with squared radius $\Delta^2(B)$, and refinement uses the square-root-free accuracy request established for boxed names in CCL-1. Separately, every finite refinement can encode a displayed non-zero rational sample of that box. No root is first extracted from the boxes.

For a rational function F/G, the Paper-I algorithm first computes and cancels the displayed gcd. It then returns the pairwise-coprime squarefree multiplicity bases of the reduced numerator and denominator. A numerator factor of multiplicity m supplies winding $+m$; a denominator factor of multiplicity m supplies winding $-m$. Constant factors contribute no finite sites, linear factors supply exact rational terminals, and higher factors supply independently shrinking tracks. The CCL wrapper applies the same terminal/track conversion to both signs and retains the Paper-I cross-separation certificates.

Definition 1.1 (CCL site observation)**.** A CCL site observation is one of two finite records.

1. An *exact observation* stores a displayed Gaussian rational, its direct polynomial evaluation certificate, and either its terminating CCL word or the distinguished exact-zero tag.
2. A *track observation* stores one Paper-I winding-one track, its current rational box, and one persistent arithmetic cursor whose quantitative candidates are members of that track's initial box.

The site index, multiplicity, sign, and separation witnesses remain fields of the surrounding Paper-I record; the CCL attachment does not recompute them.

Lemma 1.2 (a winding track is a progressive enclosure)**.** *Let (B_n) be a Paper-I winding-one track. Taking the lower-left anchor of B_n as centre and $\Delta^2(B_n)$ as squared radius defines a progressive enclosure process. A request at positive target τ returns a nested box whose ball radius is strictly below τ, and every later candidate remains a member of the earlier box.*

Proof. Use the track's finite refinement operation with the square-root-free request $r(\tau) = \min(1, \tau/4)$. Since $0 < r(\tau) \leq 1$, $r(\tau)^2 \leq r(\tau) < \tau$. The track theorem supplies both box nesting and the requested diagonal bound. Box membership is the possibility predicate, so nesting proves persistence without choosing a point from the boxes. □

Formal verification:
`CCL.TrackProcess.toProcess`.

Theorem 1.3 (persistent request for one tracked site)**.** *Initialize a joint state at a higher site's initial box B_0 and the empty arithmetic cursor. Given a later state with box B, a positive rational ε, and $r \in \mathbb{N}$, one finite request returns:*

1. *a box $B' \subseteq B$ with $\Delta^2(B') \leq \varepsilon^2$;*
2. *an exposed raw word extending the previous exposed word by exactly r bits; and*
3. *a quantitative cylinder of pairwise Hermitian projective quadrance at most ε^2.*

More explicitly, let $\widehat{w}$ be the internally committed word, including the finite unexposed buffer, let $M_{\widehat{w}}$ be its ordinary parser matrix, and let τ_q be the current projective tail chart. For any two admitted sidecar candidates t, u,

$$\chi^2\big(M_{\widehat{w}}\tau_q(t), M_{\widehat{w}}\tau_q(u)\big) \le \varepsilon^2, \qquad \Delta^2(B') \le \varepsilon^2. \tag{1.2}$$

Every admitted parser image is still an explicit rational candidate of B_0; hence the box and cylinder retain the same finite site owner.

Proof. First ask the cursor for r further bits. This changes the exposed prefix but not its originating process. Refine only the finite process state in the resulting quantitative mode. This second operation changes neither exposed nor buffered bits, parser phase, nor projective chart.

Write A_q for that chart matrix. The finite matrix $M_{\widehat{w}}A_q$ has non-zero determinant, so its explicitly calculated transport coefficient K_q is positive. Request sidecar squared radius $\varepsilon^2/(4K_q)$. Two admitted candidates in the resulting ball obey $\mathrm{N}(t-u) \le 4\rho$ by the two-leg squared-triangle estimate. Affine Hermitian projective quadrance is at most this norm, and transport through $M_{\widehat{w}}A_q$ contributes at most the factor K_q. This proves the first inequality of (1.2). Independently, the winding-track request gives $B' \subseteq B$ and the second inequality.

The exact transition invariant is preserved by both finite operations. Therefore every parser image still reconstructs to a candidate of B_0. The track index never changes, so this is also the claimed ownership certificate. □

Formal verification:
`CCL.PersistentTrack.State.request_cylinder_accurate;CCL.PersistentTrack.State.candidate_in_initial_box`.

Corollary 1.4 (finite CCL-wrapped FTA and RFAP)**.** *The finite polynomial FTA returns CCL site observations for exactly its degree-weighted zero sites. The finite rational-function argument principle returns them for every reduced zero and pole site, preserving signed multiplicity, pairwise separation, and*

$$\mathrm{wind}_{\partial B}(F/G) = \deg F_{\mathrm{red}} - \deg G_{\mathrm{red}}. \tag{1.3}$$

Exact sites retain terminating rational CCL data. Higher sites satisfy Theorem 1.3. At any request they may also return the complete canonical encoding of a displayed non-zero sample in B'; consecutive samples move by squared norm at most the earlier box diagonal.

Proof. Run Paper I once. Mapping its finite factor-observation lists is bounded recursion. A linear observation already includes direct evaluation at its displayed rational root, so attach the finite encoder output or exact-zero tag. A higher observation retains its existing track, applies the preceding lemma, and initializes the persistent state of Theorem 1.3. The executable box anchor supplies the optional canonical sample, while box membership bounds consecutive anchor movement.

No CCL operation changes the Paper-I indices or their multiplicity, separation, and winding fields. Applying the same attachment to the reduced numerator and denominator lists therefore preserves the outer identity (1.3). Every construction is a finite map over those displayed lists. □

Formal verification:
`CCL.Paper.polynomialRootStreams;CCL.PolynomialRootStreams.NonzeroResult.mem_rootIndices;CCL.PolynomialRootStreams.NonzeroResult.rootRequest_contract;CCL.Paper.rationalFunctionArgumentPrincipleWithCCL`.

1.1 Finite CCL prefixes for robust sharpness reserves

RFAP-2 proves a different finite box theorem. Given a displayed rational τ satisfying

$$\frac{3}{4} \le \tau < 1, \qquad \tau^2 - 12\tau + 9 < 0, \tag{1.4}$$

a scale T, and a run length N, it computes one proper rational box B and N strictly scale-increasing consecutive-best windows. Every displayed rational member of B violates the proposed threshold in every

one of those windows. The following adapter adds finite CCL observations without entering the RFAP dependency closure.

Theorem 1.5 (prefix-coherent finite sharpness reserve)**.** *From τ, T, N satisfying (1.4), a terminating calculation returns the RFAP run and box B, a displayed $q \in B \setminus \{0\}$, and its complete finite CCL word $v = \mathrm{enc}(q)$. For any supplied*

$$s \le T, \qquad n \le N, \qquad b \le |v|,$$

the corresponding stage exposes the same box B, the first n certified windows, and the word $w_b = v \restriction b$. It satisfies

$$\mathsf{Cyl}(w_b, q), \qquad |w_b| = b,$$

and every exposed window lies beyond s, is consecutive best for every displayed member of B, and violates τ there.

If $s \le s' \le T$, $n \le n' \le N$, and $b \le b' \le |v|$, then the later stage extends the earlier one:

$$B \subseteq B, \qquad w_b \preccurlyeq w_{b'}, \qquad (W_0, \dots, W_{n-1}) \preccurlyeq (W_0, \dots, W_{n'-1}). \tag{1.5}$$

Accordingly a stateful request for a new scale, additional windows, and additional bits computes the componentwise extension (1.5). It adds exactly the requested numbers while they fit in the declared finite reserve, and saturates at T, N, and the terminal length $|v|$ otherwise.

Proof. Run the robust RFAP construction once at the final values T, N. Properness of B gives a nonzero displayed sample by checking its lower-left corner and, only if that corner is zero, using its upper-right corner. Convert this sample coefficientwise to a Gaussian-integer fraction and run the terminating CCL encoder once. Define every word and window list by finite list `take`. The cylinder claim is the defining prefix predicate for the encoder output. Monotonicity in (1.5) is the elementary fact that a shorter `take` is a prefix of a longer one; box refinement is reflexivity. The uniform consecutive-best and violation claims are inherited directly from the RFAP box certificate, and its final scale bound implies every smaller exposed scale bound. Clipping each request by a natural-number minimum proves the saturation clauses. □

Formal verification:
`CCL.ApproximationSharpness.reserve;CCL.ApproximationSharpness.ReservedTower.Stage.cylinder;CCL.ApproximationSharpness.ReservedTower.Stage.uniformConsecutive;CCL.ApproximationSharpness.ReservedTower.Stage.uniformViolation;CCL.ApproximationSharpness.ReservedTower.stage_monotone;CCL.ApproximationSharpness.ReservedTower.Stage.extend_extends;CCL.Paper.finiteApproximationSharpnessTower`.

Here $\mathsf{Cyl}(w_b, q)$ is a *CCL prefix cylinder*: it says that the finite encoder word of the displayed rational q begins with w_b. The error–denominator cylinders used inside the best-approximation proof are finite inequalities on approximation pairs and are unrelated objects. The reserve is also deliberately finite. Rebuilding it with a larger final scale or run length may select a different rational source and a different CCL word; the theorem makes no completed-source claim across those separate calculations.

2 Gosper-style arithmetic from finite predicates

Every completed input digit or CCL block is a homographic substitution. A unary operation is represented by

$$H(x) = \frac{\alpha x + \beta}{\gamma_0 x + \delta}, \qquad M_H = \begin{pmatrix} \alpha & \beta \\ \gamma_0 & \delta \end{pmatrix}.$$

When an input block has matrix A, consumption is the finite update

$$M_H \longmapsto M_H A.$$

For output, let $\mathrm{Tail}_w(t)$ be the rational predicate produced by the finite parser after input prefix w, and let $\mathrm{Branch}_b(y)$ be the rational predicate for output bit b in the current output phase. The tail predicate also records that every denominator used by H is non-zero. Emitting b is justified by proving the implication

$$\mathrm{Tail}_w(t) \Longrightarrow \mathrm{Branch}_b\big(H(t)\big) \tag{2.1}$$

for rational t. At an incomplete CCL sector code, Tail_w is a finite disjunction indexed by the finite list Σ_q. After conjoining denominator non-vanishing, ordered inequalities can be multiplied by the positive squared norms of those denominators. Hence (2.1) reduces to finitely many rational polynomial implications. No named collection of all tails is needed.

Binary arithmetic uses the bihomographic state

$$H(x,y) = \frac{axy + bx + cy + d}{exy + fx + gy + h}.$$

Consuming a block from one input precomposes in that variable. Addition, subtraction, multiplication, and division arise from the corresponding initial coefficient tuples. The remainder of this section supplies the redundant branch semantics, squared-error transport, terminal padding, and finite margin needed for a productivity theorem; none of those additions changes the canonical encoder or prefix semantics of CCL-1.

2.1 Why the canonical branch test is insufficient

The finite rational encoder may decide the half-open comparison $\mathrm{N}(z) < 2$ or $2 \le \mathrm{N}(z)$ exactly. A transducer receiving only progressive input names cannot do so uniformly. At a point with $\mathrm{N}(z) = 2$, every finite enclosure may also contain rational points on both sides. Any algorithm which emits the unique canonical branch after inspecting only that finite information gives the same answer on two compatible refinements for which opposite answers are required.

The same obstruction occurs when a generic arithmetic operation happens to have an exact rational result. Opaque names for one chosen solution of $X^2 - 2 = 0$ do not themselves supply the algebraic certificate $x^2 = 2$. Their product can therefore be certified to arbitrary accuracy as 2 without the arithmetic transducer ever being able to decide that a terminal marker is justified. Exact termination is available when the extra equality certificate is supplied; it is not demanded from the numerical names alone.

2.2 Redundant safe sectors

For $s \in \mathcal{R}_8$, put

$$D_s(z) \ :\Longleftrightarrow\ \mathrm{N}(z - s) < \frac{1}{2}. \tag{2.2}$$

These predicates overlap. They are used only by the arithmetic output controller; the disjoint predicate Sect_s and the canonical finite encoder remain exactly as in CCL-1.

Lemma 2.1 (annulus-free safe inversion)**.** *If $D_s(z)$ and $z \neq s$, then*

$$\mathrm{N}\left(\frac{1}{\gamma(z - s)}\right) > 1. \tag{2.3}$$

If a rational enclosure lies strictly inside the axis-aligned unit cell centred at s, then every realization satisfies D_s.

Proof. The strict cell inequalities give

$$|\operatorname{Re}(z - s)| < \frac{1}{2}, \qquad |\operatorname{Im}(z - s)| < \frac{1}{2},$$

and hence $\mathrm{N}(z - s) < 1/4 + 1/4 = 1/2$. Since $\mathrm{N}(\gamma) = 2$,

$$\mathrm{N}\left(\frac{1}{\gamma(z - s)}\right) = \frac{1}{2\,\mathrm{N}(z - s)} > 1.$$

Every comparison is a cross-multiplied rational inequality. Notice that the non-overlapping unit cells alone would merely move the old ambiguity to their common edges; the overlapping predicates (2.2), rather than ownership of a grid tile, are what remove that ambiguity. □

Formal verification:
`CCL.RedundantSectors.safe_of_openCell;CCL.RedundantSectors.postInversion_norm_gt_one`.

The completed-block contraction proof in CCL-1 uses its upper annulus bound only through the finite sector-contraction calculation. Replacing that derived fact by (2.2) gives the following stronger statement.

Theorem 2.2 (redundant completed-block contraction)**.** *Suppose two displayed values z, z' use the same representative s, satisfy*

$$1 \le \mathrm{N}(z), \quad 1 \le \mathrm{N}(z'), \qquad D_s(z), \quad D_s(z'),$$

and use the same displayed radial block. For each displayed value use the affine inverse tail when its residual is non-zero, and use the formal projective tail ω when its residual is zero. Then the corresponding completed block of radial norm L obeys

$$\chi^2(A_{\lambda,s}t, A_{\lambda,s}t') \le \frac{2L}{(1+L)^2}\chi^2(t,t') \le \frac{1}{2}\chi^2(t,t'). \tag{2.4}$$

with the evident projective reading when either tail is ω. Hence any two finite projective realizations of the same n redundant completed blocks satisfy

$$\chi^2(P,Q) \le 2^{-n}. \tag{2.4a}$$

Thus crossing the old upper annulus boundary incurs no approximation error, and an exact residual incurs no exceptional failure of contraction. Both are exact redundant representation choices.

Proof. If the residual is non-zero, Equation (2.3) supplies the strict tail lower bound used in (7.14), while $1 \le \mathrm{N}(z)$ supplies the output-weight lower bound used in (7.15). If it is zero, the block acts on $\omega = [1:0]$ and reconstructs the displayed representative exactly. The same weight estimate proves the affine–ω case; the ω–ω numerator is zero. The one-block half bounds then iterate by structural induction on the supplied finite list of block realizations, proving (2.4a). The determinant calculation and all remaining inequalities in the affine–affine case are unchanged from the completed-block contraction theorem of CCL-1. □

Formal verification:
`CCL.RedundantBlockContraction.Tail.contraction_half`.

Proposition 2.3 (hypothesis-free redundant-block semantics)**.** *For every rational normalized value y, every displayed radial factor λ, and every representative s, with no annulus or safety hypothesis, the homogeneous decoder identity is*

$$A_{\lambda,s}[1 : \gamma(y-s)] \sim [\lambda y : 1], \tag{2.4b}$$

where λy is the pre-block value.

Proof. Multiply the displayed matrix by the column $[1 : \gamma(y-s)]$. Its two coordinates have the same non-zero rational scalar multiple as $[\lambda y : 1]$. No inequality is used. □

The lower-norm hypotheses in Theorem 2.2 are therefore contraction hypotheses, not exactness hypotheses. An initial or low-chart digit crossing the old lower annulus boundary is still exact by Proposition 2.3; its uncertainty is simply left to the sidecar. A non-zero safe residual has inverse-tail norm greater than one by (2.3), while a zero residual uses ω, so the following loop is normalized in either case.

Lemma 2.4 (finite projective transport)**.** *For $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ put*

$$C(M) = 2\big(\mathrm{N}(a) + \mathrm{N}(b) + \mathrm{N}(c) + \mathrm{N}(d) + 1\big), \qquad K(M) = \frac{C(\operatorname{adj} M)^2}{\mathrm{N}(\det M)}.$$

For every displayed M with non-zero determinant and all valid projective points P, Q, direct determinant and weight estimates give

$$\chi^2(MP, MQ) \le K(M)\chi^2(P,Q). \tag{2.4c}$$

Consequently, if P, Q realize the same n later certified redundant blocks, then

$$\chi^2(MP, MQ) \le K(M)2^{-n}. \tag{2.4d}$$

Proof. The alternating numerator in Hermitian projective quadrance is multiplied by $\det M$. Applying $\operatorname{adj} M$ to each image and expanding the four Gaussian-rational terms gives

$$\mathrm{N}(\det M)W(P) \le C(\operatorname{adj} M)W(MP)$$

for $W([p : q]) = \mathrm{N}(p) + \mathrm{N}(q)$. Multiplying the two weight inequalities and clearing only positive rational denominators proves (2.4c). Apply the n-block estimate (2.4a) and then (2.4c) to obtain (2.4d). □

Here M may contain every already committed unary digit and the one annulus-free initial or low-chart sector. Since M is a finite ordinary decoder prefix, $K(M)$ is an explicitly computed positive rational. Thus a fixed early loss of contraction changes only the finite bit budget: later normalized blocks drive its transported uncertainty below every positive rational request. No error symbols are added to the CCL word.

This suffix is not merely hypothetical. After the one-time initial/low chart prefix, every finite run of whole controller chunks has a finite proof-sidecar trace. Unary normalizations and residual chart changes are neutral trace steps; every live normalized sector decision contributes its actual ordinary decoder block. Reading a final candidate backwards through the trace gives a projectively coherent realization of precisely that finite block list. Equation (2.4d) therefore applies to every pair of its admitted candidates. This trace is separate from the executable one-bit cursor and does not change its output type.

Lemma 2.5 (redundant cover of the normalized loop)**.** *The eight sector-contraction cases proved in CCL-1 give*

$$\{z : 1 \le \mathrm{N}(z) \le 2\} \subseteq \bigcup_{s \in \mathcal{R}_8} D_s. \tag{2.5}$$

Consequently the loop region has the redundant finite cover

$$\{z : 1 < \mathrm{N}(z)\} \subseteq \{z : 2 < \mathrm{N}(z)\} \cup \bigcup_{s \in \mathcal{R}_8} D_s. \tag{2.6}$$

Proof. The eight finite sector cases give (2.5). Split a point of norm greater than one by the rational comparison with two: the high branch is the first member of (2.6), and the other branch lies in the annulus covered by (2.5). □

Lemma 2.6 (explicit redundant-sector margin)**.** *The eight rational cases strengthen the sector estimate to*

$$1 \le \mathrm{N}(c) \le \frac{13}{6} \quad \Longrightarrow \quad \mathrm{N}(c - s(c)) \le \frac{7}{16}, \tag{2.7}$$

where $s(c)$ is the row selected at the rational centre. The weighted square-root-free inequality

$$\mathrm{N}(u + v) \le 16\,\mathrm{N}(u) + \frac{16}{15}\,\mathrm{N}(v) \tag{2.8}$$

holds for all $u, v \in \mathbb{Q}(i)$.

Proof. Check (2.7) in the eight rational rows of the sector table. Equation (2.8) is the expansion of the non-negative quadrance $\mathrm{N}(15u - v)$. □

Lemma 2.7 (bounded redundant loop decision)**.** *A squared-error ball $\mathcal{B}(c, \rho)$ with $\rho < 1/480$ and $1 \le \mathrm{N}(c)$ admits a bounded decision. If $\mathrm{N}(c) \le 13/6$, every realization lies in $D_{s(c)}$. Otherwise every realization has norm greater than 2. The tests use the exact overlap*

$$\frac{13}{6} - \frac{32}{15} = \frac{1}{30}, \qquad \frac{1}{480} = \frac{1/30}{16}. \tag{2.9}$$

Proof. If $\mathrm{N}(c) \le 13/6$, apply (2.7) at the centre and (2.8) to the centre error to obtain one common $D_{s(c)}$ certificate. Otherwise (2.9) gives $(16/15)2 + 16\rho < \mathrm{N}(c)$, so (2.8) in the reverse direction gives $2 < \mathrm{N}(z)$. All eight representative checks and both centre comparisons are bounded finite searches. Equality at $\mathrm{N}(z) = 2$ therefore selects a safe sector rather than blocking. □

2.3 Terminal unary padding and zero

Definition 2.8 (non-return terminal)**.** If a residual is exactly zero, its inverse branch uses the projective tail ω. A *non-return terminal* is the finite cursor state which answers later requests by extending the unfinished unary run rather than by announcing a terminal marker. The already committed sector matrix is retained, and unary scaling fixes ω.

Lemma 2.9 (terminal-padding contraction)**.** *For an affine tail satisfying $2^r < \mathrm{N}(t)$,*

$$\chi^2(t,\omega) = \frac{1}{1+\mathrm{N}(t)} < \frac{1}{1+2^r}. \tag{2.10}$$

Let M be the already emitted finite decoder product. Its explicit adjugate weight estimate gives a positive rational coefficient C_M such that

$$\chi^2(Mt, M\omega) \le \frac{C_M}{1+2^r}. \tag{2.11}$$

The exact tail ω makes the left side zero. Thus every finite padding request has a certified dyadically shrinking cylinder, although the arithmetic procedure never announces termination.

Proof. The first equality is the definition of Hermitian projective quadrance at ω. Apply the finite transport estimate Lemma 2.4 to obtain (2.11). The exact projective tail gives zero directly. □

Corollary 2.10 (pairwise terminal-padding cylinder)**.** *If two residual candidates r, r' are both admitted by the small ball which justifies padding length q, their homogeneous tails are each projectively equivalent to the corresponding affine-or-ω witness. The square-root-free two-leg Hermitian-projective-quadrance estimate and (2.11) give the pairwise bound*

$$\chi^2(M[1:\gamma r], M[1:\gamma r']) \le \frac{4C_M}{1+2^q}. \tag{2.11a}$$

Proof. Each residual candidate is within the displayed small quadrance of zero. Apply terminal-padding contraction to both affine-or-ω tails, then use the square-root-free two-leg Hermitian-projective-quadrance inequality. It contributes the factor four. □

Proposition 2.11 (initial-low zero padding)**.** *Zero has an analogous initial-low no-return name. After the initial low bit and $r+1$ unary ones its finite predicate is*

$$2^r\,\mathrm{N}(z) < 1,$$

and its affine Hermitian projective quadrance from zero is below 2^{-r}.

Proof. The unary predicate itself is $2^r\,\mathrm{N}(z) < 1$, which is exactly the claimed affine Hermitian-projective-quadrance bound after dropping the positive denominator $1+\mathrm{N}(z)$. No later sector block is needed. □

These conventions permit subtraction of two opaque equal names to remain productive without deciding equality. They extend only progressive arithmetic semantics; the canonical finite encoder and its terminal marker remain unchanged.

Lemma 2.12 (finite residual small-or-apart decision)**.** *Let a residual ball have centre c, squared error ρ, and let $\theta > 0$. If $\rho < \theta/6$, a rational comparison returns one of*

$$2(\rho + \mathrm{N}(c)) < \theta, \qquad \text{or} \qquad 0 < \frac{\mathrm{N}(c)}{2} - \rho \le \mathrm{N}(z) \quad \text{for every realization } z.$$

The first alternative justifies more unary padding; the second justifies affine inversion. With $\theta = 2^{-(r+1)}$, the returned certificate also supports r padding bits. Neither branch decides whether the residual is zero.

Proof. Test the first strict rational inequality. If it holds, the squared-triangle estimate gives $\mathrm{N}(z) < \theta$ for every realization. If it fails and $\mathrm{N}(c) \le 2\rho$, then $\theta \le 2(\rho + \mathrm{N}(c)) \le 6\rho < \theta$, an impossibility. Therefore $0 < \mathrm{N}(c)/2 - \rho$, and the reverse squared-triangle estimate gives the displayed lower bound for every realization. With $\theta = 2^{-(r+1)}$, multiplicativity of N gives the inverse-tail bound. Exact zero uses ω, so neither branch decides equality. □

2.4 Squared-error sidecars

Definition 2.13 (squared-error ball)**.** For $c \in \mathbb{Q}(i)$ and $\rho \in \mathbb{Q}_{\geq 0}$, the squared-error ball $\mathcal{B}(c, \rho)$ is the finite predicate

$$\mathrm{N}(z - c) \leq \rho, \qquad z \in \mathbb{Q}(i). \tag{2.12}$$

Here ρ is already a squared error. A rational box becomes such a ball by taking its lower-left anchor as c and its squared diagonal as ρ.

Lemma 2.14 (addition and subtraction of squared-error balls)**.** *If $x \in \mathcal{B}(a, \rho)$ and $y \in \mathcal{B}(b, \sigma)$, then*

$$\begin{aligned} x + y &\in \mathcal{B}(a + b,\ 2(\rho + \sigma)), \\ x - y &\in \mathcal{B}(a - b,\ 2(\rho + \sigma)). \end{aligned} \tag{2.13}$$

Proof. Apply $\mathrm{N}(u + v) \leq 2(\mathrm{N}(u) + \mathrm{N}(v))$ to $(x - a) + (y - b)$ and to $(x - a) - (y - b)$. □

Lemma 2.15 (multiplication of squared-error balls)**.** *If $x \in \mathcal{B}(a, \rho)$ and $y \in \mathcal{B}(b, \sigma)$, then*

$$xy \in \mathcal{B}(ab,\ 4(\rho\,\mathrm{N}(b) + \mathrm{N}(a)\sigma) + 2\rho\sigma)\,. \tag{2.13a}$$

Proof. Expand $xy - ab = (x - a)b + a(y - b) + (x - a)(y - b)$. Two applications of the squared triangle inequality give the displayed bound, using multiplicativity of N for each product. □

Lemma 2.16 (apartness-certified division)**.** *Suppose $x \in \mathcal{B}(a, \rho)$, $y \in \mathcal{B}(b, \sigma)$, and finite evidence supplies $\delta > 0$ with*

$$y \in \mathcal{B}(b, \sigma) \Longrightarrow \delta \leq \mathrm{N}(y). \tag{2.14}$$

Then $b \neq 0$ and

$$\frac{x}{y} \in \mathcal{B}\left(\frac{a}{b}, \frac{2(\rho\,\mathrm{N}(b) + \mathrm{N}(a)\sigma)}{\delta\,\mathrm{N}(b)}\right). \tag{2.15}$$

Proof. The certificate at $y = b$ gives $\delta \leq \mathrm{N}(b)$, hence $b \neq 0$. Now

$$\frac{x}{y} - \frac{a}{b} = \frac{(x - a)b - a(y - b)}{yb};$$

the numerator is bounded by the addition/subtraction lemma and the denominator quadrance is at least $\delta\,\mathrm{N}(b)$. These are finite rational calculations. Division is deliberately partial without (2.14): a zero or pole outcome must remain explicit rather than being hidden by an invalid inverse. □

Definition 2.17 (progressive enclosure process)**.** A progressive enclosure process consists of finite states q, a ball $\mathcal{B}_q$, a finite-state possibility predicate $P_q(z)$, and a request operation

$$R(q, \eta), \qquad \eta \in \mathbb{Q}_{>0},$$

such that

$$P_q(z) \Longrightarrow z \in \mathcal{B}_q, \qquad P_{R(q,\eta)}(z) \Longrightarrow P_q(z), \qquad \rho_{R(q,\eta)} < \eta. \tag{2.16}$$

Each state contains one displayed rational witness satisfying its current possibility predicate. This is data returned by the finite procedure, not an existence assertion subsequently eliminated by choice.

Lemma 2.18 (candidate apartness becomes ball apartness)**.** *Let w be the displayed witness in a process state with ball $\mathcal{B}(c, \rho)$. If $\delta \leq \mathrm{N}(w)$ and $\rho < \delta/12$, then*

$$\frac{\delta}{8} \leq \mathrm{N}(z) \quad \textit{for every rational } z \in \mathcal{B}(c, \rho). \tag{2.16a}$$

Proof. Both w and z are in $\mathcal{B}(c, \rho)$. The squared triangle inequality first bounds $\mathrm{N}(w - z)$ by 4ρ. Its reverse form then gives $\mathrm{N}(w) \leq 2\,\mathrm{N}(z) + 2\,\mathrm{N}(w - z)$, so $\delta < 2\,\mathrm{N}(z) + 2\delta/3$. Weakening the resulting $\delta/6 < \mathrm{N}(z)$ to the stated $\delta/8$ keeps the certificate rational and non-strict. □

Consequently one finite refinement turns candidate-level non-zero evidence into the whole-ball apartness required for affine division. Persistence of the possibility predicate keeps that construction available after every later refinement.

Lemma 2.19 (weighted quadrance inequality)**.** *For every rational $m > 0$ and $u, v \in \mathbb{Q}(i)$,*

$$\mathrm{N}(u+v) \le (m+1)\,\mathrm{N}(u) + \frac{m+1}{m}\,\mathrm{N}(v), \tag{2.16b}$$

Proof. Expand the non-negative quadrance $\mathrm{N}(mu - v)$ and rearrange. □

Lemma 2.20 (finite preparation of the loop lower bound)**.** *Suppose every current candidate has norm at least $L > 1$. Put $m = 2/(L-1)$ and $\kappa = (m+1)/m$. A refinement below*

$$\frac{L-\kappa}{m+1}$$

has a rational centre of norm at least one. Hence a common target below this number and $1/480$ supplies the hypotheses of the bounded redundant loop decision in one finite request.

Proof. The chosen m has $\kappa < L$. If the refined centre had norm below one, apply (2.16b) to the displayed witness written as centre plus error. The requested error bound would make the witness norm strictly smaller than L, contrary to its candidate certificate. □

Proposition 2.21 (exact residual rebasing)**.** *After a safe sector has been emitted, rebase the finite process at the state on which that decision was made. Every later candidate is therefore still admitted by the safe state. Exact translation by the representative produces the residual process $r = z - s$ and preserves the explicit strict upper bound already present in the safe-ball inequality. If residual apartness is returned, one further calculated refinement makes the whole residual ball apart and division constructs*

$$t = \frac{\gamma^{-1}}{r} = (\gamma r)^{-1}.$$

The retained residual upper bound supplies a uniform rational lower bound $L > 1$ for every candidate t.

Proof. Persistence gives the first assertion. Translation is exact and preserves the proved upper bound. Candidate-to-ball apartness and apartness-certified division construct t. Multiplicativity of quadrance turns the residual upper bound into the asserted lower bound for t, after which the loop-preparation lemma applies. Thus this is an exact chart change, not accumulated numerical error. □

Each process also carries a rational $M \ge 0$ bounding the squared norm of every displayed centre. This is finite persistence data, not a selected completed complex value. Binary operations use pairs of operand states; for example, the new finite predicate for addition says that the displayed rational candidate is $x + y$ for two candidates satisfying the two current operand predicates. The implication in (2.16) is then preserved componentwise.

Proposition 2.22 (closed operation requests and centre bounds)**.** *For a desired output squared error $\tau > 0$, let M_x, M_y be the two centre bounds. The following common operand requests suffice:*

operation	*operand squared-error target η*
$x \pm y$	$\tau/4$
xy	$\dfrac{\tau}{\tau + 4(M_x + M_y) + 2}$
x/y	$\dfrac{\tau\delta^2}{4(M_x + M_y + 1)}$ *when* $\delta \le \mathrm{N}(y)$.

(2.17)

The transported centre bounds are respectively

$$2(M_x + M_y), \qquad M_x M_y, \qquad M_x/\delta. \tag{2.18}$$

Thus the output of each operation is itself a progressive finite enclosure process and may be used as an operand of another operation.

Proof. Addition and subtraction follow immediately from (2.13). For multiplication put $K = 4(M_x + M_y) + 2$. The displayed request satisfies $0 < \eta < 1$ and $\eta K < \tau$, so (2.13a) is strictly below τ. For division, the numerator in (2.15) is at most $2\eta(M_x + M_y)$ and its denominator is at least δ^2; the strict inequality $2(M_x + M_y) < 4(M_x + M_y + 1)$ gives the result. Applying the same estimates to the rational centres gives (2.18). Componentwise refinement proves persistence, so no search is hidden in any request. □

Lemma 2.23 (affine inverse-chart sensitivity)**.** *For $z, w \neq s$,*

$$\mathrm{N}\left(\frac{1}{\gamma(z-s)} - \frac{1}{\gamma(w-s)}\right) = \frac{\mathrm{N}(z-w)}{2\,\mathrm{N}(z-s)\,\mathrm{N}(w-s)}. \tag{2.19}$$

Proof. Subtract the two fractions, use $\mathrm{N}(\gamma) = 2$, and use multiplicativity of quadrance. □

The lemma explains why a scalar ε^2 alone is insufficient: an affine inversion additionally needs residual separation. When separation has not yet been proved, the controller retains a projective enclosure and emits unary padding rather than forming an uncertified affine quotient.

2.5 Bihomographic operations and productivity

Definition 2.24 (initial bihomographic arithmetic states)**.** The four arithmetic operations start from the following bihomographic coefficient tuples:

	a	b	c	d	e	f	g	h
$x + y$	0	1	1	0	0	0	0	1
$x - y$	0	1	−1	0	0	0	0	1
xy	1	0	0	0	0	0	0	1
x/y	0	1	0	0	0	0	1	0.

(2.20)

Input blocks are consumed by finite precomposition in the relevant variable. Between block consumptions, (2.13)–(2.18) compute an output enclosure. The controller refines the input, using the closed targets (2.17), until one redundant output predicate holds for the entire enclosure. It then emits that exact output bit or sector code, substitutes the corresponding inverse output matrix into the bihomographic state, and continues.

The executable realization may transport the two input balls and the current output chart directly instead of retaining all eight coefficients at every step. The two descriptions are propositionally linked. Every candidate of the progressive addition, subtraction, multiplication, or apartness-certified division process is projectively equal to the value of the corresponding initial row of (2.20). Moreover, for an arbitrary bihomographic state T, input decoder matrices L, R, and output matrix O, the finite coefficient updates satisfy the following identity.

Lemma 2.25 (bihomographic transition identity)**.** *For every bihomographic state T, input decoder matrices L, R, and output matrix O,*

$$\big(O \circ (T \circ (L, R))\big)(x, y) = O\big(T(Lx, Ry)\big). \tag{2.21}$$

For an emitted decoder block A, take $O = \mathrm{adj}(A)$.

Proof. Expand both sides as homogeneous matrix products and use associativity of finite matrix multiplication. No division or normalization is involved. □

Thus direct enclosure transport and Gosper's matrix bookkeeping are certified views of the same arithmetic transition; productivity is proved for the former while (2.21) proves the exact substitution interpretation.

Definition 2.26 (exact cursor invariant)**.** The exact cursor invariant uses a projective interpretation of each quantitative mode. If v is a candidate of the mode, put

q	$\tau_q(v)$
initial	$[v : 1]$
initial-low with count m	$[(\gamma^m)^{-1}v : 1]$
regular with count k	$[\gamma^k v : 1]$
residual	$[1 : \gamma v]$.

(2.22)

Let $\widehat{w}$ be the exposed raw prefix followed by the unexposed remainder of its one current finite output chunk. If q_0 is the original finite process state, the invariant is

$$P_q(v) \Longrightarrow \exists z \in \mathbb{Q}(i) \quad P_{q_0}(z) \quad \text{and} \quad M_{\widehat{w}}\tau_q(v) \sim [z:1]. \tag{2.23}$$

The existential witness here is returned by the finite candidate proof. It is not a root or a selected completed value.

Theorem 2.27 (closure of progressive enclosure arithmetic)**.** *Given two progressive enclosure processes, addition, subtraction, and multiplication construct progressive enclosure processes. Division does so from a positive denominator-apartness certificate. Every positive rational squared-error request is answered by one finite request to each operand.*

Proof. Use the product finite state and possibility predicate described in (2.16). The three ball-transport lemmas and the closed-operation request proposition prove strict output accuracy and the new centre bounds. Componentwise refinement proves persistence, while each product state carries the displayed pair of operand witnesses. □

Theorem 2.28 (finite totality of the redundant controller)**.** *At a normalized loop state, one refinement below* $1/480$ *returns either a sound unary branch or a sound redundant sector. At a residual state, one refinement below* $\theta/6$ *returns either a sound residual bound below* θ *or positive apartness sufficient for affine inversion. The initial chart is likewise prepared by a fixed finite refinement. None of these decisions tests equality of an opaque result.*

Proof. At a loop state whose displayed centre has norm at least one, request radius below $1/480$ and apply the bounded loop-decision lemma; its ball-level certificate is sound for every candidate admitted by the refined finite state. For a requested residual target θ, request radius below $\theta/6$ and apply the residual small-or-apart lemma. In the small branch, taking $\theta = 2^{-(r+1)}$ supplies the projective loop-tail witness for r unary padding bits; in the apart branch the positive lower bound supplies the exact hypothesis of (2.15). Each construction is a finite function call on rational data, and no completed stream or completed complex value is formed.

For the initial chart, first refine below $1/480000$. If the displayed centre has norm below one, emit the initial-low selector and one low-unary bit; the weighted estimate (2.8) gives the rescaled candidate bound $\mathrm{N}(\gamma z) < 43/20 < 13/6$. If the centre has norm at least one, the overlapping decision following (2.9) emits either an exponent-zero low block or the first high-unary bit. In a subsequent low state, the retained $43/20$ bound and (2.16b), with $m = 1000$, keep the next sufficiently refined centre strictly below $13/6$. A centre of norm at least one therefore emits a safe sector, while a centre below one emits another low-unary bit and preserves the same bound. Exact zero follows this latter branch forever. The ordinary loop and residual modes then use the two decisions above. □

Theorem 2.29 (productive raw cursor and unchanged grammar)**.** *The controller decisions assemble into a finite-state raw-bit cursor. From every state and for every* $n \in \mathbb{N}$*, bounded recursion returns an extension containing exactly* n *further bits without retracting the old prefix. Parsing every committed whole chunk with the ordinary CCL-1 parser gives the syntax phase recorded by the controller. Thus the accuracy sidecar adds no symbol to the raw CCL grammar.*

Proof. Every emitted chunk is non-empty: initial-low and sector chunks may be buffered, but the public cursor exposes them one bit at a time. Structural recursion on n consequently proves both prefix extension and the exact length increase n. A finite case analysis of the initial-high, initial-low, loop, and residual modes shows that each chunk takes the syntax-only raw parser from its stored phase to the phase stored by the next mode. Associativity of finite-list append lifts this local fact through the one-bit buffer and the same structural recursion, proving the committed-word parser invariant. □

Theorem 2.30 (exact semantics of the productive arithmetic cursor)**.** *Equation (2.23) holds after every finite request of the productive cursor for addition, subtraction, multiplication, or apartness-certified division. Consequently every internally committed extension has the exact semantics of the ordinary raw parser for the original arithmetic process.*

After a redundant completed block, a non-zero residual r *has the exact affine tail* $(\gamma r)^{-1}$*, while* $r = 0$ *has the exact projective tail* ω*. More generally,* $\mathrm{N}(r) < 2^{-(q+1)}$ *gives either the exact* ω *tail or an exact affine tail of norm greater than* 2^q *and Hermitian projective quadrance at most* $(1 + 2^q)^{-1}$ *from* ω*.*

Proof. For exact semantics, the unary branches use the two cancellation identities

$$\gamma^{k+1}(\gamma^{-1}v) = \gamma^k v, \qquad (\gamma^{m+1})^{-1}(\gamma v) = (\gamma^m)^{-1}v.$$

A sector branch uses the hypothesis-free homogeneous identity (2.4b). Translation makes its next residual candidate exactly $v - s$. Small-residual padding leaves $[1 : \gamma(v - s)]$ unchanged. In the apart branch, normalizing the affine inverse by the already emitted unary count gives

$$[\gamma^k(\gamma^k)^{-1}(\gamma r)^{-1} : 1] \sim [1 : \gamma r],$$

so the silent chart change is exact before the next ordinary loop step. For a normalized-loop sector outcome, the loop-domain lower certificate and the selected whole-ball safe-sector certificate apply simultaneously to every pair of actual post-emission residual candidates. Replacing their homogeneous tails by the equivalent affine-or-ω tails therefore specializes (2.4) directly to that live controller transition. Non-zero determinants preserve projective validity, allowing these local equivalences to compose with the previous parser product. Associativity of finite-list append then lifts the result through the buffer and induction on the requested bit count, proving (2.23). Finally, (2.10) proves the displayed no-return bound. Thus productivity and exactness do not depend on recognizing an exact rational result. □

Formal verification:
`CCL.ProgressiveTransducer.Named.add_productive;CCL.ProgressiveTransducer.Named.mul_productive;CCL.ProgressiveTransducer.Named.div_productive`.

3 Computing exponential and logarithm as CCL names

The objective of this section is an operation in the CCL representation, not merely a separate routine producing rational Taylor approximants. An arithmetic CCL name consists of the productive raw-word cursor of Section 2 together with its rational enclosure sidechannel. We construct a total operator

$$\mathsf{ExpName} : \mathsf{CCLName} \longrightarrow \mathsf{CCLName}$$

and certified logarithm operators on their stated domains. Thus a user can continue requesting ordinary CCL bits of the exponential result, while each request also carries enough finite rational accuracy to justify those bits. The sidechannel is essential: the theorem does not claim that a bare raw word, with no quantitative enclosure information, suffices to schedule all input refinements.

The arithmetic interface permits these operations to be constructed without adding completed real or complex points. We follow the finite recurrences used by Collins [5], but change their logical role. An approximation is a finite program with a natural-number request argument. On a displayed input n it computes finite data

$$(a_n, \rho_n) \tag{3.1}$$

consisting of the Gaussian rational a_n and rational squared allowance ρ_n; for displayed $n \le m$ the same program proves

$$\mathrm{N}(a_m - a_n) \le \rho_n; \tag{3.2}$$

and for every displayed rational $\tau > 0$ it computes an index $r(n, \tau) \ge n$ with $\rho_{r(n,\tau)} < \tau$. Subscripts here are a request schema for running that finite program, not a stored infinite stream. The possibility predicate at index n is not a named set or an intersection: it is the constructive proposition that returns a finite witness $m \ge n$ and an equality between the supplied rational value and a_m. Thus (3.2) puts every such witnessed later sample in the displayed ball $\mathcal{B}(a_n, \rho_n)$.

This approximation process is the internal accuracy sidechannel from which the output CCL controller makes its finite decisions. This is the meaning of *error* in this section. It is a checked finite tail-error certificate between rational samples. We neither consume an infinite sequence as input nor measure distance to a silently postulated analytic value. The words "exponential" and "logarithm" first name the following recurrence interfaces. Their productivity proofs do not require a completion. The analytic-identification layer is instead phrased as finite operational equality: two programs are equal when every positive rational accuracy request computes later displayed tails which are pairwise that close. This retains the intended function laws without silently adjoining completed non-rational values.

3.1 The local exponential recurrence

For $x \in \mathbb{Q}(i)$ satisfying $\mathrm{N}(x) \le 1$, define the finite Horner terms

$$E_{0,j}(x) = 1, \qquad E_{n+1,j}(x) = 1 + \frac{x}{j} E_{n,j+1}(x), \qquad e_n(x) = E_{n+1,1}(x). \tag{3.3}$$

Only natural recursion on the displayed depth n occurs. In particular, (3.3) is not notation for an infinite continued expression. Direct quadrance estimates prove

$$\mathrm{N}(e_m(x) - e_n(x)) \le \frac{10}{4^n} \qquad (n \le m), \tag{3.4}$$

and, for two unit-disk inputs,

$$\mathrm{N}(e_n(x) - e_n(y)) \le 16\,\mathrm{N}(x - y). \tag{3.5}$$

The proof bounds every finite inner Horner tail and uses $j^{-2} \le 1/4$ for $j \ge 2$; induction then propagates the displayed quarter-contraction. Consequently $e_n(x)$, with allowance $10/4^n$, is a finite approximation process in the sense of (3.1)–(3.2).

For a progressive input process, take at stage n the rational witness of its nth calculated refinement and evaluate (3.3) to depth n. Two such input witnesses at stages $n \le m$ have quadrance at most $4/4^n$. Combining this fact with (3.4)–(3.5) and the square-root-free two-leg inequality gives the deliberately factored allowance

$$\rho_n^{\exp} = \frac{2(10 + 16 \cdot 4)}{4^n}. \tag{3.6}$$

No large decimal numeral is required by the executable proof.

Theorem 3.1 (the local exponential is CCL-computable)**.** *For every arithmetic CCL input name whose candidates at the displayed starting state satisfy* $\mathrm{N}(x) \le 1$*, equations (3.3)–(3.6) construct an arithmetic CCL output name. Every positive rational squared-error request terminates, and the output's ordinary CCL cursor returns every requested finite number of further bits without retracting its previous prefix. Its exact-cursor certificate identifies every admitted parser tail with a rational candidate of the calculated exponential output process at that same finite state.*

Proof. Use (3.6) as the output name's process radius. A bounded scan of successive powers of $1/4$ finds the first displayed index below the requested target. The scan fuel is the binary length of the computed rational ceiling of the scale-to-target quotient plus one. Equations (3.4)–(3.6) prove persistence and ball containment, so this finite process is a valid accuracy sidechannel for a CCL name. Feed that process to the unchanged redundant CCL controller of Section 2. Theorem 2.30 returns every requested output bit, and its exact-cursor induction proves that the committed ordinary raw prefix and the exponential enclosure describe the same finite output state. This last step is what turns the Horner recurrence into computation *with the CCL*, rather than leaving it as an unrelated rational approximation algorithm. □

Formal verification:
`CCL.ProgressiveExp.productive`.

3.2 The local positive-real logarithm recurrence

For a rational x in the overlapping chart

$$\frac{1}{2} \le x \le 2, \qquad z = \frac{x-1}{x+1}, \qquad w = z^2, \tag{3.7}$$

direct cross-multiplication gives $-1/3 \le z \le 1/3$ and $0 \le w \le 1/9$. Put

$$L_{0,j}(w) = 1, \qquad L_{n+1,j}(w) = 1 + \frac{2j-1}{2j+1} w L_{n,j+1}(w), \qquad \ell_n(x) = 2z L_{n,1}(w). \tag{3.8}$$

Again every term is a finite rational Horner calculation. The coefficient and chart bounds imply

$$\left|\tfrac{2j-1}{2j+1}w\right|^2 \le \frac{1}{4}, \qquad \mathrm{N}(\ell_m(x)-\ell_n(x)) \le \frac{1}{4^n} \quad (n \le m). \tag{3.9}$$

A denominator-cleared comparison of two displayed inputs in the same chart also gives the finite sensitivity estimate

$$\mathrm{N}(\ell_n(x)-\ell_n(y)) \le 64(x-y)^2. \tag{3.10}$$

For progressive chart inputs, the witness movement allowance is again $4/4^n$. Combining (3.9)–(3.10) therefore gives

$$\rho_n^{\log} = \frac{2(1+64\cdot 4)}{4^n}. \tag{3.11}$$

Theorem 3.2 (the local positive-real logarithm is CCL-computable)**.** *Suppose every candidate of the displayed arithmetic CCL input state is real and has real coordinate in* $[1/2, 2]$*. Equations (3.7)–(3.11) construct an arithmetic CCL output name for the local logarithm recurrence. Every positive rational accuracy request terminates, and its ordinary CCL cursor returns every requested finite number of further bits with exact same-state parser semantics.*

Proof. The chart hypothesis applies to every explicitly selected later witness. Equations (3.9)–(3.11) prove the approximation fields, and the same bounded quarter-power scan answers every rational accuracy request. Use that process as the output CCL name's accuracy sidechannel and start the unchanged controller of Section 2. Its productivity theorem supplies every requested ordinary CCL bit, while exact-cursor semantics tie all admitted parser tails to candidates of the logarithm process at the same state. Thus the result is a local CCL-to-CCL operation, not merely a list of rational Horner values. □

Formal verification:
`CCL.ProgressiveLog.productive`.

3.3 Automatic finite range and branch preparation

Lemma 3.3 (finite exponential range reduction)**.** *Exponential needs no branch information. At the current input state let* c *and* r *be the rational centre and squared radius, and compute*

$$B = 2\big(r + \mathrm{N}(c)\big). \tag{3.12}$$

Every current candidate x *satisfies* $\mathrm{N}(x) \le B$*. A bounded quarter-power scan computes* $k \in \mathbb{N}$ *with*

$$\frac{B}{4^k} < 1. \tag{3.13}$$

Hence every candidate of the exactly scaled process $x/2^k$ *lies in the unit disk. Apply the local CCL construction there and then apply CCL multiplication as exact repeated squaring exactly* k *times. Thus the global finite CCL name is defined by*

$$\mathsf{ExpName}(X) = \mathrm{Sq}^k\big(\mathsf{ExpLocal}(X/2^k)\big)\,; \tag{3.14}$$

then every input to ExpLocal *lies in its unit-disk chart.*

Proof. The squared triangle inequality gives (3.12). A bounded scan of the finite list $B, B/4, \dots, B/4^k$ finds (3.13). Since division by 2^k divides quadrance by 4^k, the scaled process lies in the unit disk. Local exponentiation followed by exactly k CCL squarings gives (3.14). This is an operational range reduction, not an equality between independently named completed values. □

Definition 3.4 (positive-real logarithm certificate)**.** A positive-real logarithm certificate consists of a rational $L > 0$ and the proof

$$P_q(x) \Longrightarrow \mathrm{Im}(x) = 0 \ \text{ and } \ L \le \mathrm{Re}(x) \tag{3.15}$$

for every candidate of the displayed state.

Lemma 3.5 (finite dyadic logarithm chart preparation)**.** *Given a positive-real logarithm certificate, refine the state once to squared radius below*

$$\left(\frac{L}{16}\right)^2. \tag{3.16}$$

If c is the new rational ball centre, a displayed candidate witness and the square comparison in (3.16) prove

$$\frac{L}{2} \le \operatorname{Re}(c), \tag{3.17}$$

so $\operatorname{Re}(c) > 0$ without extracting any common point from the process.

A bounded binary scan now computes a direction $d \in \{\mathsf{up}, \mathsf{down}\}$ and exponent $k \in \mathbb{N}$ whose rational dyadic factor s puts the centre safely inside the overlap:

$$\frac{3}{4} \le s\operatorname{Re}(c) \le \frac{3}{2}, \qquad s = 2^k \text{ or } s = 2^{-k}. \tag{3.18}$$

For values above one, the scan fuel is $\lceil \operatorname{Re}(c) \rceil$; below one it is $\lceil 1/\operatorname{Re}(c) \rceil$. The first adjacent power certificate is adjusted by at most one exponent to obtain (3.18). Equations (3.16)–(3.18) give

$$s\frac{L}{16} \le \frac{1}{4}, \tag{3.19}$$

and therefore every candidate in the refined ball satisfies

$$\operatorname{Im}(sx) = 0, \qquad \frac{1}{2} \le \operatorname{Re}(sx) \le 2. \tag{3.20}$$

This proves the candidate-wise local chart required by (3.7), including inputs on dyadic boundaries. The rational ball supplies the deviation bound; the separate certificate (3.15), rather than the disk geometry, proves that all process candidates are real. No claim is made that every irrelevant geometric point of the two-dimensional disk is real.

Proof. The displayed candidate witness and (3.16) imply (3.17) by the reverse squared triangle estimate. The bounded upward or downward power-of-two scan gives adjacent dyadic bounds, adjusted by at most one exponent to obtain (3.18). Applying the same estimate to every candidate gives (3.19) and (3.20). All comparisons are rational and the scan has the displayed finite fuel. □

Definition 3.6 (global positive-real logarithm name)**.** Generate the local CCL name $\ell_n(2)$ once and combine it by CCL arithmetic. Writing $\mathsf{LogTwoName} = \mathsf{LogLocal}(2)$, define

$$\mathsf{LogName}(X) = \begin{cases} \mathsf{LogLocal}(X/2^k) + k\mathsf{LogTwoName}, & d = \mathsf{down}, \\ \mathsf{LogLocal}(2^k X) - k\mathsf{LogTwoName}, & d = \mathsf{up}. \end{cases} \tag{3.21}$$

Here each occurrence of "log" denotes its displayed finite recurrence inside a productive CCL name. Multiplication by the displayed integer k and addition are the already proved finite CCL operations. Hence automatic positive-real charting does not leave the representation. No exact non-rational constant $\log 2$ is introduced.

3.4 A certified complex branch chart

Definition 3.7 (local complex logarithm recurrence)**.** The same finite recurrence works over Gaussian rationals once its reduced argument is supplied directly. For $z \in \mathbb{Q}(i)$ with

$$\mathrm{N}(z) \le \frac{1}{9}, \qquad w = z^2, \tag{3.22}$$

define $L_{n,j}(w)$ exactly as in (3.8) and put $\ell_n^{\mathrm{C}}(z) = 2zL_{n,1}(w)$.

Lemma 3.8 (local complex logarithm bounds)**.** *For $n \le m$ and two reduced arguments z, v in the disk (3.22),*

$$\mathrm{N}\big(\ell_m^{\mathrm{C}}(z) - \ell_n^{\mathrm{C}}(z)\big) \le \frac{5}{4^n}, \qquad \mathrm{N}\big(\ell_n^{\mathrm{C}}(z) - \ell_n^{\mathrm{C}}(v)\big) \le 64\,\mathrm{N}(z - v). \tag{3.23}$$

The deliberately loose factor 5 avoids square roots and keeps every bound in the Gaussian-rational field. Synchronizing finite input refinement and Horner depth therefore gives the progressive allowance

$$\rho_n^{\mathrm{C}\log} = \frac{2(5 + 64 \cdot 4)}{4^n}. \tag{3.24}$$

Proof. The bounds in (3.23) follow by the same finite Horner induction as (3.9)–(3.10), now using multiplicativity of Gaussian quadrance. The loose factor 5 avoids square roots and keeps every allowance rational. Combining tail and input movement gives (3.24) by the two-leg inequality. □

Definition 3.9 (certified complex logarithm chart)**.** A one-chart logarithm for an arithmetic CCL input Y consists of the following displayed finite data. Choose $a \in \mathbb{Q}(i)$; for every finite input state prove a fixed rational lower quadrance $\lambda > 0$ for every candidate of $aY + 1$; and at the starting state prove, conservatively for every independently admitted pair of candidates y_1, y_2, that

$$\mathrm{N}\left(\frac{ay_1 - 1}{ay_2 + 1}\right) \le \frac{1}{9}. \tag{3.25}$$

The independent pair is required because finite enclosure arithmetic may refine the two occurrences of Y separately. One calculated refinement below a fixed rational fraction of λ converts candidate-wise separation into apartness of the entire displayed denominator ball. Hence progressive division can construct the reduced-argument name

$$Z = \frac{aY - 1}{aY + 1}.$$

Finally supply a progressive correction name C. The correction is the branch choice. It may itself be non-rational and is therefore a finite-request name, not a postulated completed constant. The safe-chart interface does not guess a cut, an anchor, or an integer branch index. Safety and productivity alone permit an arbitrary correction. To certify that it is genuinely logarithmic, we additionally require the finite operational branch condition

$$\mathsf{ExpName}(C) \simeq \mathsf{Exact}(a^{-1}), \qquad a \neq 0, \tag{3.26a}$$

where $\simeq$ is the finite-request equivalence defined below. This is insensitive to the expected $2\pi i$ branch shifts but excludes an unrelated correction.

Definition 3.10 (charted complex logarithm name)**.** For a certified chart, prepare the division $Z = (aY - 1)/(aY + 1)$ and put

$$\mathsf{ComplexLogName}(Y; a, C) = \mathsf{ComplexLogLocal}(Z) + C. \tag{3.26}$$

Prepared division, the local Gaussian recurrence, and correction addition are all CCL operations, so this is an arithmetic CCL output name.

Definition 3.11 (CCL-computable operation)**.** An operation on arithmetic CCL names is *CCL-computable* when, for every rational $\tau > 0$ and $n \in \mathbb{N}$, a finite request returns an output state of squared radius below τ and exactly n further ordinary CCL bits. The same request must return an exact-cursor certificate: every admitted tail, acted on by the ordinary parser for the committed word and pending finite chunk, is projectively equivalent to an explicitly witnessed candidate of that output state.

Theorem 3.12 (global exponential is CCL-computable)**.** *For every arithmetic CCL name X, the name $\mathsf{ExpName}(X)$ of (3.14) is CCL-computable.*

Proof. Equations (3.12)–(3.14) reduce exponential to the local process and a finite recursion of CCL multiplication nodes. Consequently range reduction does not leave the CCL representation: scaling, local exponential, and every successive squaring produce arithmetic CCL names. Refine the resulting process at τ, then invoke Theorem 2.30 at that same state for the n bits and its exact-parser certificate. □

Theorem 3.13 (positive-real logarithm is CCL-computable)**.** *If X carries the positive-real certificate (3.15), then* $\mathsf{LogName}(X)$ *of (3.21) is CCL-computable. Its preparatory refinement, dyadic exponent, local chart proof, and correction coefficient are displayed finite data.*

Proof. The dyadic chart-preparation lemma reduces the input to (3.7). The local logarithm theorem, the independently generated LogTwoName, and finite CCL arithmetic construct (3.21). Refine its output process at τ and invoke the arithmetic cursor theorem at that state for the requested bits and exact semantics. □

Theorem 3.14 (charted complex logarithm is CCL-computable)**.** *If X carries the certified complex chart data (3.25)–(3.26a), then* $\mathsf{ComplexLogName}(X; a, C)$ *is CCL-computable.*

Proof. Candidate apartness becomes whole-ball apartness after one calculated refinement, so progressive division constructs Z. Equations (3.22)–(3.24) construct its local Gaussian logarithm, and CCL addition adds the supplied correction. The generic output refinement and exact cursor theorem answer the joint accuracy-and-bit request. Denominator preparation, Horner evaluation, correction addition, and cursor execution are all finite. □

Corollary 3.15 (certified exponential-minus-logarithm composition)**.** *Let X and Y be progressive Gaussian-rational input processes, and give Y the positive-real certificate (3.15). Define, operationally,*

$$\mathsf{GName}(X, Y) = \mathsf{ExpName}(X) - \mathsf{LogName}(Y). \tag{3.27}$$

For every rational $\tau > 0$ and $n \in \mathbb{N}$, a finite request returns a rational output ball of squared radius below τ, exactly n exposed ordinary CCL bits, and the same-state exact-cursor certificate of the preceding theorem. The same conclusion holds when Y instead carries the complex chart data (3.25)–(3.26) and $\mathsf{LogName}(Y)$ *is replaced by* $\mathsf{ComplexLogName}(Y; a, C)$*.*

Proof. Apply the already constructed exponential and certified logarithm processes, their finite progressive subtraction, and then the generic combined request at target τ and count n. The same proof, replacing LogName by ComplexLogName and using (3.25)–(3.26), gives the charted complex version. □

Formal verification:
`CCL.TranscendentalOutput.exp_ccl_computable`.

Formal verification:
`CCL.TranscendentalOutput.log_ccl_computable`.

Formal verification:
`CCL.TranscendentalOutput.complex_log_ccl_computable`.

Positive-real logarithm is not a claim about a global complex branch. The complex extension above likewise does not manufacture branch information: it consumes the explicit scale, denominator clearance, reduced-disk proof, and correction. Subject to such chart data at every logarithm node, finite expression trees built from arithmetic, exponential, and logarithm remain productive by composition. Corollary 3.15 provides one such g node. This is the precise domain condition needed before using repeated g constructions to specify further transcendental operations.

3.5 Finite operational equality and function laws

Definition 3.16 (finite operational equality)**.** For approximation programs $A = (a_n, \rho_n)$ and $B = (b_n, \sigma_n)$, write $A \simeq B$ when, from any displayed starting indices p, q and any rational $\tau > 0$, a terminating comparison computes $r \geq p$ and $s \geq q$ such that

$$\mathrm{N}(a_m - b_n) < \tau \qquad \text{for every displayed } m \geq r \text{ and } n \geq s. \tag{3.28}$$

All quantifiers here range over natural indices and Gaussian-rational samples. The comparison is data: it returns the synchronizing indices.

Proposition 3.17 (operational equality is an equivalence relation)**.** *Finite operational equality is reflexive, symmetric, and transitive.*

Proof. Reflexivity uses identical indices, and symmetry swaps the returned pair. For transitivity, request quarter of the target from each comparison, synchronize the two intermediate indices, and apply the two-leg quadrance inequality. Every index adjustment is a maximum of finitely displayed natural numbers. □

Applying the canonical nested finite sampling path of an arithmetic CCL name therefore gives a finite operational equality for finite-request CCL programs.

The raw process accuracy field alone is not enough for this conclusion: independently requested refinements may follow unrelated branches. We therefore attach the following finite certificate.

Definition 3.18 (coherent arithmetic CCL name)**.** A coherence certificate assigns each finite state s a nonnegative rational semantic allowance $\eta(s)$ and a positive rational request modulus $R(\tau)$ such that every two explicitly possible rational readings u at s and v at t satisfy

$$\mathrm{N}(u - v) \le 4\big(\eta(s) + \eta(t)\big), \tag{3.28a}$$

and, whenever a positive process request r obeys $r < R(\tau)$,

$$\eta(\mathsf{refine}(s, r)) < \tau. \tag{3.28b}$$

Both R and the refined state are computed finite data. Exact rational names have allowance zero.

Proposition 3.19 (coherence yields operational equality)**.** *Any two starting states of one coherent process give operationally equal CCL names. Coherence is preserved by addition, negation, subtraction, and multiplication.*

Proof. Use (3.28b) twice with a sufficiently small rational fraction of the target, then apply (3.28a). Addition and negation use the squared triangle estimate. Multiplication uses the squared-error product polynomial in the two semantic allowances and their rational centre bounds, followed by the closed request of (2.17). □

Every displayed-sample approximation inherits a certificate from its later-sample estimate. The certificates are preserved by the preceding proposition. The local and range-reduced exponential, repeated squaring, the positive logarithm and its dyadic correction, and a complex logarithm with a coherent explicit branch correction all carry this certificate. No common completed value is selected.

Lemma 3.20 (local exponential at zero)**.** *For every displayed finite depth n,*

$$E_{n,j}(0) = 1, \qquad e_n(0) = 1 \tag{3.29}$$

and therefore $\mathsf{ExpLocal}(0) \simeq \mathrm{exact}(1)$.

Proof. Direct induction through (3.3) gives $E_{n,j}(0) = 1$, hence $e_n(0) = 1$. Identical displayed samples satisfy every operational tolerance. □

Theorem 3.21 (local exponential addition law)**.** *For $x, y \in \mathbb{Q}(i)$ with $\mathrm{N}(x) \le 1$, $\mathrm{N}(y) \le 1$, and $\mathrm{N}(x+y) \le 1$, let the left approximation display $e_n(x)e_n(y)$ and the right approximation display $e_n(x + y)$. Then*

$$\mathsf{ExpLocal}(x + y) \simeq \mathsf{ExpLocal}(x)\mathsf{ExpLocal}(y). \tag{3.30}$$

The product on the right is the actual stateful CCL multiplication process.

Proof. The constructive proof does not appeal to uniqueness of an infinite series. Bounded induction identifies (3.3) exactly with the finite polynomial $\sum_{k \le n} x^k/k!$: expose each nested Horner contribution, rewrite its consecutive denominator product as the executable finite factorial, and add the final monomial. A finite Cauchy-product calculation and the binomial theorem identify every coefficient through the displayed truncation degree. The remaining finite triangular tail satisfies, at the synchronized odd depth $2k + 1$,

$$\mathrm{N}\big(e_{2k+1}(x)e_{2k+1}(y) - e_{2k+1}(x + y)\big) \le \frac{16}{4^k}. \tag{3.30a}$$

The independent tail allowances on both sides are also explicit, so a bounded quarter-power scan computes the synchronization indices required by (3.28) for every displayed $\tau > 0$. Thus (3.30) is an operational law,

not merely a pointwise estimate at one depth. Moreover, the formal statement is now connected to the actual arithmetic expression tree. A separate finite bridge follows the stateful `Named.mul` process to one displayed multiplication state, bounds its two component radii, and compares every later product reading with the pointwise product approximation. Hence the proved left side of (3.30) is literally the CCL multiplication of the two progressive local exponential names, rather than a product stream merely wrapped in a fresh name. The local exponential also respects operational equality of arbitrary unit-disk input names, by applying the 16δ finite Horner sensitivity bound at one synchronized depth and absorbing the two finite truncation tails. □

Lemma 3.22 (congruence of finite repeated squaring)**.** *For every displayed $k \in \mathbb{N}$, coherent names $X \simeq Y$ satisfy*

$$\mathsf{SquareIterate}(k, X) \simeq \mathsf{SquareIterate}(k, Y). \tag{3.30d}$$

Proof. Canonically square the displayed samples of any approximation and transport its later-sample allowance through the two-factor product estimate. For a coherent CCL name, a computed request makes both operand readings of the actual stateful self-multiplication close to one canonical sampled reading; this compares `Named.mul` X X with that canonical square approximation. Finite induction on k proves the result through the actual repeated self-multiplication tree. □

Lemma 3.23 (finite local logarithm reconstruction bound)**.** *For $\mathrm{N}(z) \le 1/9$, put*

$$\ell_n^{\mathrm{C}}(z) = 2\sum_{j=0}^{n} \frac{z^{2j+1}}{2j+1}, \qquad P_n(z) = \sum_{r=0}^{2n+2} \frac{(\ell_n^{\mathrm{C}}(z))^r}{r!}.$$

Then the executable estimate

$$\mathrm{N}\left(P_n(z) - \frac{1+z}{1-z}\right) \le \frac{118}{4^n}. \tag{3.30b}$$

Proof. Finite coefficient differentiation and substitution prove that the coefficients of P_n agree through degree $2n+2$ with those of $1 + 2z + 2z^2 + \cdots$. A non-negative finite coefficient majorant bounds the remaining terms. Substitution of $\mathrm{N}(z) \le 1/9$ gives (3.30b). □

Theorem 3.24 (displayed rational exponential–logarithm inverse)**.** *For a displayed rational chart input $y \in \mathbb{Q}(i)$, put $z = (ay-1)/(ay+1)$. If the chart certificate gives $\mathrm{N}(z) \le 1/9$ and the anchor (3.26a) supplies q_n with $\mathrm{N}(q_n - a^{-1}) < 4^{-n}$, then*

$$\mathrm{N}\big(P_n(z)q_n - y\big) \le \frac{2\big(50 + 118\,\mathrm{N}(a^{-1})\big)}{4^n}. \tag{3.30c}$$

where 50 is a uniform quadrance bound for $P_n(z)$. Consequently

$$\mathsf{Exp}(\mathsf{ComplexLog}(y; a, C)) \simeq y \tag{3.31}$$

for the displayed rational input.

Proof. The chart certificate proves the denominator non-zero, and direct Gaussian-rational cancellation gives $(1+z)/(1-z) = ay$. Combine the local reconstruction bound with the anchor error and the uniform bound 50 to get (3.30c). A bounded quarter-power scan supplies the indices for (3.31). Every sample is rational; the finite round-trip theorem compares controller resampling with the originating approximation, so the emitted CCL name is operationally equivalent to exact y. No completed logarithm value is asserted. □

Theorem 3.25 (arithmetic congruence)**.** *Operational equality of coherent names is a congruence for addition and multiplication. Multiplication is operationally commutative and associative. For every displayed k,*

$$\mathsf{SquareIterate}(k, X)\mathsf{SquareIterate}(k, Y) \simeq \mathsf{SquareIterate}(k, XY). \tag{3.30e}$$

Proof. Equal-index sampled sums and products give common rational approximations for coherent names. Comparing actual stateful arithmetic with these approximations proves two-input congruence for addition and multiplication, and proves commutativity and associativity of multiplication, without selecting completed operand values. Reassociation and induction on k give (3.30e). □

Theorem 3.26 (automatic exponential addition for rational inputs)**.** *For all displayed $x, y \in \mathbb{Q}(i)$,*

$$\mathsf{ExpName}(\text{exact}\, x)\mathsf{ExpName}(\text{exact}\, y) \simeq \mathsf{ExpName}(\text{exact}(x+y)). \tag{3.30g}$$

The three automatic range exponents need not agree.

Proof. Dyadic exponent reconciliation is finite for exact Gaussian-rational data. Let $D_0(z) = z$ and $D_{k+1}(z) = D_k(z)/2$, and put $E_k(z) = \mathsf{SquareIterate}(k, \mathsf{LocalExp}(D_k(z)))$ whenever the displayed local unit-disk certificate holds. Applying (3.30) to $D_{k+1}(z) + D_{k+1}(z) = D_k(z)$ proves $E_{k+1}(z) \simeq E_k(z)$; finite induction permits any larger admissible exponent. Taking the finite maximum of three independently supplied exponents and using (3.30e) therefore gives

$$E_{k_x}(x)E_{k_y}(y) \simeq E_{k_s}(x+y) \tag{3.30f}$$

for exact rational x, y and any three admissible k_x, k_y, k_s. For the public automatic entry point, actual multiplication of the exact dyadic scale and exact input is operationally equivalent to their displayed rational product. Local exponential congruence and (3.30d) transport this bridge through the whole automatically selected squaring tree, proving (3.30g). □

Theorem 3.27 (local exponential addition for coherent names)**.** *If the possible finite readings of coherent names X, Y, and $X + Y$ lie in the unit disk, then*

$$\mathsf{LocalExp}(X)\mathsf{LocalExp}(Y) \simeq \mathsf{LocalExp}(X+Y). \tag{3.30h}$$

Proof. For arbitrary coherent progressive inputs, the local addition law is now complete. The actual sum is transported to an equal-index sampled wrapper; its possible finite readings inherit the actual sum's unit-disk certificate. At one computed odd depth the exact polynomial bound (3.30a) is instantiated with the two displayed operand readings. Explicit dynamic Horner tails compare actual multiplication of the local exponential outputs with the dynamic sum-Horner name. A second finite scan synchronizes that name with local exponential evaluation on the sampled-sum wrapper, after which local exponential congruence returns to the actual sum process. No completed operand values are selected. □

Theorem 3.28 (operational exponential laws)**.** *For coherent progressive CCL inputs,*

$$\mathsf{ExpName}(X)\mathsf{ExpName}(Y) \simeq \mathsf{ExpName}(X+Y), \tag{3.30i}$$

and $\mathsf{ExpName}(\text{exact}\, 0) \simeq \text{exact}\, 1$. *The total exponential also respects operational equality. Every synchronization index is computed from finite certificates and the requested positive rational tolerance.*

Proof. The same law now holds for the total automatic transform. The three range reductions for X, Y, and $X + Y$ may select different exponents. Their finite maximum determines three explicit halving chains. Each chain is proved operationally equivalent to the canonical sampler multiplied by the same displayed inverse dyadic. Distributivity at those rational samples, the common-scale unit-disk bound, (3.30h), and finite repeated-squaring congruence give (3.30i). This is the actual automatic process on all three sides, not an idealized stream and not a theorem about selected completed values. The exact-zero specialization is also connected to the finite identity sample:

$$\mathsf{ExpName}(\text{exact}\, 0) \simeq \text{exact}\, 1. \tag{3.30j}$$

The common-refinement argument, local Horner sensitivity, and repeated squaring congruence also transport operational equality through the total exponential. □

Corollary 3.29 (operational inverse and non-vanishing)**.** *For every coherent name X,*

$$\mathsf{ExpName}(-X)\mathsf{ExpName}(X) \simeq 1, \qquad \mathsf{ExpName}(X)\mathsf{ExpName}(-X) \simeq 1. \tag{3.30k}$$

In particular $\mathsf{ExpName}(X) \not\simeq 0$.

Proof. Addition has operational commutativity, associativity, zero, and additive inverse laws; multiplication has commutativity, associativity, unit, zero, and both distributivity laws. The exponential addition law and additive inverse law give (3.30k). Exact zero and exact one fail the comparison (3.28) already at the displayed squared tolerance $1/2$. If $\mathsf{ExpName}(X) \simeq 0$, multiplication congruence with the independently constructed $\mathsf{ExpName}(-X)$ would make that same product equivalent both to exact zero and to exact one, contradicting this finite separation. No completed exponential value or global apartness oracle is asserted. □

Formal verification:
`CCL.TranscendentalCoherence.automaticExp_add`.

Formal verification:
`CCL.ProgressiveCalculusCongruence.exponential_addition`.

Lemma 3.30 (Möbius reconstruction sensitivity)**.** *If* $\mathrm{N}(r), \mathrm{N}(s) \le 1/9$*, then*

$$\mathrm{N}\left(\frac{1+r}{1-r} - \frac{1+s}{1-s}\right) \le \frac{81}{4}\,\mathrm{N}(r-s). \tag{3.31c}$$

Proof. Direct cancellation gives $(1+r)/(1-r) - (1+s)/(1-s) = 2(r-s)/((1-r)(1-s))$. The square-root-free estimates $\mathrm{N}(1-r), \mathrm{N}(1-s) \ge 4/9$, followed by multiplicativity of quadrance, give the result. □

Lemma 3.31 (charted-input sensitivity)**.** *Put* $T_a(x) = (ax-1)/(ax+1)$*. If the displayed denominators for* x *and* y *have quadrance at least* $\delta > 0$*, then*

$$\mathrm{N}(T_a(x) - T_a(y)) \le \frac{4\,\mathrm{N}(a)}{\delta^2}\,\mathrm{N}(x-y). \tag{3.31d}$$

Proof. Put the two fractions over their common denominator. The numerator reduces to $2a(x-y)$; use the two denominator lower bounds and multiplicativity of quadrance. □

Lemma 3.32 (exact synchronized chart reconstruction)**.** *Let* y_n *be one rational candidate read from a prepared input state and set* $r_n = T_a(y_n)$*. Then*

$$\frac{1+r_n}{1-r_n}a^{-1} = y_n. \tag{3.31e}$$

The samples r_n *form a progressive reduced-input name with an explicit quarter-power allowance and inherit the chart's* $1/9$ *disk certificate.*

Proof. The charted-input sensitivity lemma transports the input coherence allowance to the r_n. The disk certificate is candidate-wise and hence applies to each displayed y_n. Equation (3.31e) is direct rational cancellation. □

Lemma 3.33 (asynchronous quotient comparison)**.** *Write* $F_a(x,y) = (ax-1)/(ay+1)$*. Suppose* $\mathrm{N}(z) \le M$ *and every displayed denominator has quadrance at least* $\delta > 0$*. Then*

$$\mathrm{N}(F_a(x,y) - T_a(z)) \le \frac{4(\mathrm{N}(a)^2 M + \mathrm{N}(a))}{\delta^2}\big(\mathrm{N}(x-z) + \mathrm{N}(y-z)\big). \tag{3.31f}$$

Consequently, for a coherent name Y*, the stateful prepared quotient is operationally equivalent to the synchronized name:*

$$R_a^{\mathrm{state}}(Y) \simeq \mathsf{Name}\big((T_a(y_n))_{n\ge 0}\big). \tag{3.31g}$$

Proof. Common-denominator cancellation and the squared triangle estimate give (3.31f). For a requested tolerance, compute a smaller coherence budget. One bounded scan drives the prepared-division request below its corresponding input request, and a second reaches a diagonal source index with the same allowance. Apply (3.31f) to the two source witnesses and the diagonal witness. This returns the finite synchronization indices required by (3.31g), without selecting a common value. □

Theorem 3.34 (anchored exponential–logarithm inverse)**.** *Let Y be a coherent arithmetic CCL name and let (a, C) be an anchored logarithm chart for it: the chart includes the displayed denominator and reduced-disk certificates, a productive correction name C, and the finite exponential anchor for a^{-1}. Then the actual CCL processes satisfy*

$$\mathsf{ExpName}(\mathsf{ComplexLog}(Y; a, C)) \simeq Y.$$

The comparison algorithm works for every requested positive rational squared tolerance and does not select a completed logarithm value.

Proof. The total automatic exponential respects operational equality between arbitrary coherent inputs. Applying (3.30i) in reverse to the actual charted-logarithm sum, then applying the branch certificate, gives

$$\mathsf{ExpName}(\mathsf{ComplexLog}(Y; a, C)) \simeq \mathsf{ExpName}(\mathsf{LocalLog}(R_a(Y)))\ \mathrm{exact}(a^{-1}). \tag{3.31a}$$

At this stage the arbitrary-input inverse obligation reduces precisely to the local charted reconstruction on the right of (3.31a); exponent splitting, branch correction, and total-transform congruence are no longer involved. The finite reconstruction below discharges that obligation. Branch changes remain explicit: they are transported only through checked overlapping finite enclosures, and a zero-discrepancy logarithm law requires a displayed alignment certificate. Nothing here defines a global single-valued logarithm.

The sharper finite Horner estimate puts every displayed local-log sample in the exponential unit disk. At reduced-sample depth n, use the finite program $P_n(r_n)$ with exponential depth $2n + 1$. The reconstruction bound (3.30b), together with the explicit truncation and input-sensitivity allowances, gives

$$\mathsf{ExpName}(\mathsf{LocalLog}(R_a(Y)))\,\mathrm{exact}(a^{-1}) \simeq \mathsf{Name}\left(\left(\frac{1+r_n}{1-r_n}a^{-1}\right)_{n\geq 0}\right). \tag{3.31b}$$

The name on the right has a closed quarter-power error schedule; it denotes a finite approximation program, not a selected limit.

The Möbius reconstruction and charted-input sensitivity lemmas transport operational equality through the last chart. One finite denominator-preparation state is projected back to its input component; at depth n the reference reads one rational candidate y_n there and emits $T_a(y_n)$. The exact synchronized reconstruction lemma gives (3.31e), and the asynchronous quotient comparison gives (3.31g). Exact reconstruction turns the scaled diagonal chart into a finite subsampling of the prepared input sampler. The explicit input-coherence certificate rebases that prepared state to the original input state. Combining this arithmetic comparison with (3.31a)–(3.31e) proves the full anchored round trip for arbitrary coherent progressive inputs,

$$\mathsf{ExpName}(\mathsf{ComplexLog}(Y; a, C)) \simeq Y. \tag{3.31h}$$

This completes the finite comparison algorithm and hence the proof. □

Formal verification:
`CCL.ComplexLogCorrectness.actualExp_chartedLog_named_equivalent`.

3.6 Finite logarithm branches and certified compositions

Definition 3.35 (anchored logarithm chart and transition)**.** An anchored logarithm chart is finite data: the rational Möbius chart, its denominator and reduced-disk certificates, a productive correction CCL, and the operational anchor needed for the exponential round trip. Given two such charts A, B over the same input, define their transition by the actual CCL subtraction tree

$$\Delta(A, B) = \mathsf{LogName}(Y; B) - \mathsf{LogName}(Y; A). \tag{3.31i}$$

Theorem 3.36 (finite logarithm branch calculus)**.** *For anchored charts A, B, C over the same coherent input,* $\mathsf{ExpName}(\Delta(A, B)) \simeq 1$ *and*

$$\Delta(A, A) \simeq 0, \qquad \Delta(A, B) + \Delta(B, A) \simeq 0, \qquad \Delta(A, B) + \Delta(B, C) \simeq \Delta(A, C).$$

Proof. First, $\mathsf{ExpName}(\Delta(A,B)) \simeq 1$. Transitions to the same chart are operationally zero, opposite transitions add to zero, and three supplied charts satisfy

$$\Delta(A,B) + \Delta(B,C) \simeq \Delta(A,C). \tag{3.31j}$$

Finite induction gives the corresponding statement for any displayed finite chain of charts. □

Theorem 3.37 (branch-safe logarithm product law)**.** *For supplied branches of X, Y, and a process operationally equivalent to the actual product XY, the discrepancy between $\log(XY)$ and $\log X + \log Y$ exponentiates to exact one.*

Proof. Expand the discrepancy using the total exponential addition and inverse laws. The three anchored inverse theorems and the supplied product-input equivalence reduce its exponential to exact one. The discrepancy remains an explicit CCL name rather than a silently chosen completed multiple of $2\pi i$. □

Corollary 3.38 (aligned logarithm product law)**.** *When a separate finite alignment certificate proves that this actual CCL discrepancy is zero, finite addition and cancellation give the stronger guarded law*

$$\mathsf{LogName}(XY;C) \simeq \mathsf{LogName}(X;A) + \mathsf{LogName}(Y;B). \tag{3.31j'}$$

Proof. The product-input equivalence and the zero discrepancy are both fields of the certificate; neither is inferred from the fact that the discrepancy has unit exponential. □

Theorem 3.39 (productivity of finite certified expressions)**.** *These operations are closed under a finite certified expression package. Each node stores its constructed coherent CCL output and an explicit natural node count. Leaves and exact rationals need no side condition; division stores denominator apartness and a logarithm node stores its anchored chart. Addition, negation, multiplication, exponential, and those guarded nodes compute their output processes recursively. For every rational $\tau > 0$ and bit count n, the expression computes one refined finite state whose ball has squared radius below τ, emits exactly n further raw CCL bits, and returns the same-state exact parser certificate. Its coherence sidecar also computes a positive rational request modulus. Thus arbitrary finite composition preserves the CCL-with-error computational contract; no semantic value is assigned to the expression outside its finite requests.*

Proof. Proceed by induction on the finite expression tree. Leaves use exact or supplied coherent names. Arithmetic nodes use progressive closure; exponential and logarithm use their CCL-computability theorems; division and logarithm consume the stored apartness or chart certificate. Compose the rational request moduli and invoke the exact cursor at the resulting state. □

Lemma 3.40 (quarter-power scan cost)**.** *Every quarter-power refinement scan used by these constructions has a closed finite cost bound. Starting at index j with non-negative scale S and positive target τ, the returned index is at most*

$$j + \left(\log_2 \lceil S/\tau \rceil + 1\right), \tag{3.31k}$$

where the logarithm and ceiling are executable operations on displayed natural and rational data.

Proof. Run the scan with exactly the displayed fuel. Repeatedly multiplying the denominator by four puts the scale below τ before the fuel is exhausted. This is bounded natural recursion, not an unbounded convergence argument. Expression nodes compose their displayed request moduli; no uniform cost independent of those finite node certificates is claimed. □

Formal verification:
`CCL.LogBranchGeometry.logarithm_product_discrepancy_unit`.

Formal verification:
`CCL.LogBranchGeometry.logarithm_product_of_alignment`.

Formal verification:
`CCL.CertifiedExpression.Expression.bitstream_and_error`.

3.7 Projective rational-function evaluation

Proposition 3.41 (finite projective rational-function evaluation)**.** *Let $f, g \in \mathbb{Q}(i)[T]$ and let X be a certified expression. Finite Horner recursion constructs coherent numerator and denominator expressions and retains the result projectively as*

$$[\mathsf{Eval}(f, X) : \mathsf{Eval}(g, X)]. \tag{3.31l}$$

This construction respects operational equality and specializes to the ordinary affine or reciprocal-affine chart whenever the corresponding displayed denominator apartness certificate is supplied.

Proof. Horner evaluation respects operational equality. At a displayed rational input it is operationally equivalent to the exact displayed polynomial value, by finite induction through multiplication and addition. No denominator decision is required for the projective pair. The ordinary affine expression $f(X)/g(X)$ is exposed only from supplied finite apartness of the denominator. Around a pole, supplied numerator apartness instead exposes the reciprocal chart $g(X)/f(X)$; denominator equivalence to exact zero together with that numerator certificate is the finite pole record. Thus rational functions compose with CCL computations without adjoining an affine infinity or completing either coordinate. □

Formal verification:
`CCL.ProjectiveRationalEvaluation.evaluate_exact`.

3.8 Finite divided differences and differential laws

The differential assertions used here do not quantify over an infinitesimal, take a limit, or first identify a function with a completed set of ordered pairs. They begin with finite syntax and an exact two-input factor.

Definition 3.42 (power divided-difference kernel)**.** For a natural exponent define

$$K_n(x, y) = \begin{cases} 0, & n = 0, \\ x^{n-1} + yK_{n-1}(x, y), & n > 0. \end{cases}$$

Lemma 3.43 (power divided difference)**.** *For every displayed natural n and every $x, y \in \mathbb{Q}(i)$,*

$$x^n - y^n = (x - y)K_n(x, y), \qquad K_n(x, x) = nx^{n-1}. \tag{3.31m}$$

Proof. Finite induction in $\mathbb{Q}(i)$ proves the first identity in (3.31m). The second equality is obtained only after the first polynomial factor has been constructed; no division by $x - y$ occurs, including on the diagonal. □

Corollary 3.44 (finite polynomial divided differences)**.** *Every displayed finite polynomial p has an executable kernel Δp satisfying*

$$p(x) - p(y) = (x - y)\Delta p(x, y), \qquad \Delta p(x, x) = p'(x), \tag{3.31n}$$

where p' is the executable finite formal derivative.

Proof. Take the coefficient-weighted sum of the power kernels. Distribute the exact factor $(x - y)$ before evaluating on the diagonal; the diagonal identity is then the finite coefficient formula for p'. □

Proposition 3.45 (finite rational-expression kernels)**.** *Signed powers, finite Laurent term lists, and a rational expression f/g with the required displayed non-zero denominator certificates have finite divided-difference kernels. Product, composition, reciprocal, and quotient kernels obey their usual exact rules.*

Proof. Negative powers use explicit Gaussian-rational inverse cancellation. Substitute the already constructed kernels into the finite product, composition, reciprocal, and quotient identities and clear the certified non-zero denominators. These identities define the derivative data; they do not appeal to an ambient analytic derivative. □

Formal verification:
`CCL.DividedDifference.polynomialKernel_diagonal`.

Theorem 3.46 (finite differential law for exponential)**.** *The finite exponential program has an exact two-input secant factorization, valid on the diagonal, whose first coefficient is the exponential program. After subtraction of the linear term, the remainder has an explicit rational bound proportional to* $\mathrm{N}(h)^2$*. The factor and remainder are productive CCL names, giving the operational law* $D\exp(x) = \exp(x)$*.*

Proof. The local exponential has an especially economical factor. Put

$$R_n(z) = E_{n,2}(z), \qquad S_n(z) = \frac{1}{2}E_{n,3}(z),$$

using the finite Horner programs of (3.3). At every displayed depth, including $z = 0$,

$$e_n(z) - 1 = zR_n(z), \qquad R_{n+1}(z) - 1 = zS_n(z), \qquad e_{n+1}(z) - 1 - z = z^2 S_n(z). \tag{3.31o}$$

Thus R_n is the executable relative exponential usually denoted exprel, but it is defined by the middle Horner program rather than by the unsafe quotient $(e_n(z) - 1)/z$. Likewise S_n is a finite second relative remainder. On $\mathrm{N}(z) \le 1$ both carry explicit quarter-power squared-error schedules and uniform rational centre bounds, so each feeds the ordinary productive CCL controller and emits every requested finite prefix. Direct induction gives $R_n(0) = 1$.

For a displayed base x and increment h, the product computation $e_n(x)e_n(h)$ therefore has exact finite secant factor

$$e_n(x)e_n(h) - e_n(x) = h\, e_n(x)R_n(h),$$

and, one depth later, exact quadratic remainder

$$e_{n+1}(x)e_{n+1}(h) - e_{n+1}(x) - he_{n+1}(x) = h^2 e_{n+1}(x)S_n(h). \tag{3.31p}$$

The displayed unit-disk bounds make the word "quadratic" quantitative in the paper's quadrance convention:

$$\mathrm{N}\big(e_{n+1}(h) - 1 - h\big) \le \mathrm{N}(h)^2, \qquad \mathrm{N}(\text{translated remainder}) \le 10\,\mathrm{N}(h)^2.$$

Since $\mathrm{N}(h)$ is already squared ordinary magnitude, these are the usual $O(|h|^2)$ remainder bounds expressed without square roots. On the diagonal $h = 0$ the displayed secant factor is exactly $e_n(x)$. The already proved finite operational addition law identifies the product computation with the direct computation of $e(x + h)$ at arbitrary requested rational accuracy. This is the constructive differential law $D\exp(x) = \exp(x)$ for the CCL-computable exponential: an exact finite factorization, a shrinking quadratic sidecar, and an operational bridge to the public exponential process. □

Theorem 3.47 (finite differential law for an anchored logarithm)**.** *On a supplied anchored chart, the local logarithm program has an exact two-input secant factorization valid on the diagonal. Its first coefficient is operationally the reciprocal program, its quadratic remainder has an explicit rational bound, and all three programs are productive CCL names. Thus* $D\log(x) = x^{-1}$ *on that chart.*

Proof. For $\mathrm{N}(u) \le 1/4$ define the displayed finite programs

$$L_n(u) = \sum_{k=0}^{n} \frac{(-u)^k}{k+1}, \qquad Q_n(u) = -\sum_{k=0}^{n} \frac{(-u)^k}{k+2}, \qquad D_n(u) = \sum_{k=0}^{n} (-u)^k.$$

All three sums are natural-number recursions over Gaussian rationals. The local logarithm sample is $uL_n(u)$, and exact finite rearrangement gives

$$L_{n+1}(u) - 1 = uQ_n(u), \qquad uL_{n+1}(u) - u = u^2 Q_n(u). \tag{3.31q}$$

The proved uniform bound $\mathrm{N}(Q_n(u)) \le 4$ therefore gives the explicit finite estimate

$$\mathrm{N}\big(uL_{n+1}(u) - u\big) \le 4\,\mathrm{N}(u)^2.$$

More fundamentally, let $F_n(u) = uL_n(u)$. The finite-polynomial divided difference constructs a two-input expression $\Delta_n(x, y)$ before any diagonal substitution, and proves

$$F_n(x) - F_n(y) = (x - y)\Delta_n(x, y), \qquad \Delta_n(u, u) = D_n(u).$$

Thus D_n is not merely a separately chosen approximation to the expected answer: it is exactly the diagonal value of the secant factor for the same finite local-log program. The derivative program has the division-free geometric certificate

$$(1+u)D_n(u) = 1 - (-u)^{n+1}. \tag{3.31r}$$

The disk hypothesis proves directly that $\mathrm{N}(1+u) \geq 1/4$ and hence $\mathrm{N}((1+u)^{-1}) \leq 4$. Consequently

$$\mathrm{N}\big(D_n(u) - (1+u)^{-1}\big) \leq 4^{-n}. \tag{3.31s}$$

The proof constructs the comparison index for every positive rational squared tolerance, so the derivative approximation is operationally equivalent to the exact displayed reciprocal; it does not appeal to an infinite geometric series. The programs L_n and Q_n separately carry the same kind of productive CCL sidecar.

For an anchored branch write $x = a(1+u)$ with $a \neq 0$. The correction that selects the logarithm branch is constant with respect to x, and therefore has zero first coefficient. Explicit inverse cancellation gives

$$(1+u)^{-1}a^{-1} = \big(a(1+u)\big)^{-1} = x^{-1}. \tag{3.31t}$$

This transports the local result to the branch-aware law $D\log(x) = x^{-1}$ without choosing a global logarithm. □

Corollary 3.48 (finite differential laws for exponential and logarithm)**.** *The exponential and every supplied anchored logarithm chart satisfy their respective operational differential laws, with exact secant factorizations and explicit quadratic-error certificates.*

Proof. Combine the preceding two theorems. □

Formal verification:
`CCL.ExponentialCalculus.expDifferential_equivalent_direct`.

Formal verification:
`CCL.LogarithmicCalculus.anchoredLogDerivative_inverse`.

The preceding differential laws extend through finite syntax, coefficient tables, and path refinements. We state each layer separately so that its data, hypotheses, and computational conclusion remain visible.

Definition 3.49 (finite first jet)**.** A first jet is a pair $(v, d) \in \mathbb{Q}(i)^2$. Its product, reciprocal, and composition operations are

$$(v,d)(w,e) = (vw, dw + ve), \qquad (v,d)^{-1} = \big(v^{-1}, -d(v^{-1})^2\big), \qquad (w,e) \circ (v,d) = (w, de). \tag{3.31u}$$

Theorem 3.50 (finite first-jet evaluation)**.** *Structural recursion evaluates finite expression syntax containing constants, one input, arithmetic, reciprocal, local exponential, and local logarithm in one structural pass, returning its displayed value and first coefficient. Jet evaluation leaves the ordinary finite-depth value unchanged and proves the coefficient rule at every node. Required local-domain conditions are decidable predicates on displayed rational subexpression values.*

Proof. Induct on the finite expression syntax. Constants and the input are immediate. Equation (3.31u) handles products, reciprocals, and composition; the two differential theorems supply the exponential and guarded logarithm nodes. Each induction branch returns the ordinary value equation together with its first-coefficient equation. □

Definition 3.51 (CCL sine and cosine)**.** For every coherent arithmetic CCL input Z, set in the already constructed total exponential and field operations

$$\sin(Z) = \frac{\mathrm{Exp}(iZ) - \mathrm{Exp}(-iZ)}{2i}, \qquad \cos(Z) = \frac{\mathrm{Exp}(iZ) + \mathrm{Exp}(-iZ)}{2}. \tag{3.31v}$$

Theorem 3.52 (sine and cosine are CCL-computable)**.** *Both domains are exactly the type of coherent CCL names; no bounded chart is required. Each output answers a requested positive rational squared error and emits exactly the requested finite bit count with the same-state parser certificate.*

Proof. Scalar multiplication, negation, addition, subtraction, and division by the displayed non-zero rational constants are CCL-computable. Compose them with the total exponential theorem in the two finite trees of (3.31v). □

Theorem 3.53 (synchronized progressive first jets)**.** *A synchronized progressive first jet has value and slope two coherent CCL names accompanied by finite indexed provenance: constants, the independent input, addition, negation, product, reciprocal, quotient, composition, total exponential, anchored logarithm, sine, and cosine are the only constructors, and each constructor computes both result channels by its displayed rule. In particular, an arbitrary pair of unrelated coherent names cannot be inserted as a derivative. Given a target $\tau > 0$, the program forms one positive rational harmonic request strictly below τ and below both channels' coherence moduli. The resulting two enclosure radii and two semantic allowances are all below τ, and both cursors return the same requested finite number of bits. This supplies a finite, bit-producing forward-differentiation program over arbitrary coherent progressive inputs; the quantitative change law is stated separately rather than inferred from the two channels alone.*

Proof. Induct through the permitted provenance constructors and apply the first-jet rule at each node. Fold a positive harmonic request through the two finite coherence moduli. The resulting radii and allowances are below the target; the persistent cursors supply the exact bit counts and parser certificates. □

Definition 3.54 (quadratic change certificate)**.** A finite certificate at increment h stores displayed b, d, o, r, F and proves

$$o - b - hd = r, \qquad \mathrm{N}(r) \le F\,\mathrm{N}(h)^2. \tag{3.31w}$$

Proposition 3.55 (finite quadratic-change calculus)**.** *Affine, sum, negation, scalar, product, apartness-guarded reciprocal, and quotient rules compute new tuples directly. For product and reciprocal the factor is the exact displayed rational ratio $\mathrm{N}(r)/\mathrm{N}(h)^2$ when $h \neq 0$, with the zero case discharged by the algebraic expansion before any division. On $\mathrm{N}(h) \le 1$, one certificate has the explicit displacement sensitivity $S = 2(\mathrm{N}(d) + F)$. If an outer certificate is supplied at the inner displayed displacement, finite substitution gives the composite factor*

$$F_{\mathrm{comp}} = 2\big(\mathrm{N}(d_{\mathrm{out}})F_{\mathrm{in}} + F_{\mathrm{out}}S_{\mathrm{in}}^2\big). \tag{3.31x}$$

The exponential and local-log bounds 10 and 4 proved above instantiate this same record. Consequently a finite composite retains an explicit rational quadratic bound, rather than only an informal big-O annotation at its leaves.

Proof. Expand (3.31w) for each arithmetic constructor and collect its exact linear and residual parts. When $h \neq 0$, the displayed rational ratio $\mathrm{N}(r)/\mathrm{N}(h)^2$ supplies the product or reciprocal factor; at $h = 0$ the algebraic identity discharges the case before division. For composition, substitute the inner identity into the outer one and apply the squared triangle estimate, obtaining (3.31x). The exponential and logarithm differential theorems supply factors 10 and 4. □

Definition 3.56 (progressive quadratic trace)**.** A progressive quadratic trace supplies a certificate $C_n = (h_n, b_n, o_n, d_n, r_n, F_n)$ at each index, one displayed geometric scale, finite later-closeness bounds for all five varying fields, and one rational bound for their centres.

Theorem 3.57 (persistent bitstreams for a quadratic trace)**.** *The trace data construct coherent channels for h, b, o, d, r and a computable joint request index. Each channel is then rooted at that exact process state and retained as an actual persistent CCL cursor. For every rational target $\tau > 0$ and finite bit count, one synchronized request returns five enclosure-and-cursor records: every root radius is below τ, every cursor extends its existing raw word by exactly the requested number of bits, its projective cylinder has the proved pairwise Hermitian-projective-quadrance bound, and all five cursors retain exact ordinary-parser semantics. The five root centres are literally the fields of C_n, and hence prove*

$$o_n - b_n - h_n d_n = r_n, \qquad \mathrm{N}(r_n) \le F_n\,\mathrm{N}(h_n)^2. \tag{3.31w'}$$

This is not a tuple of rational facts adjacent to the CCL calculation: its same-index finite observations are the roots of the live bit-emitting cursors. The constant-trace constructor recovers every exact terminating rational

certificate. A second executable witness uses the changing finite exponential approximation at the displayed input 1 *as* h_n*, proves* $h_0 \neq h_1$ *by finite rational calculation, and nevertheless satisfies the same productive bitstream-and-quadratic contract. It therefore exercises the nonterminating-input route rather than silently replacing it by an exact rational value.*

Proof. Fold one request through the five finite later-closeness moduli and root every cursor at the resulting common state. The trace certificate at that state is exactly (3.31w'). The persistent cursor theorem supplies prefix extension, length, Hermitian-projective-quadrance, and parser certificates for every channel. The constant trace is immediate. The changing exponential witness is checked by direct rational evaluation at its first two depths and then uses the same generic request construction. □

Proposition 3.58 (finite channel transport algebra)**.** *If channels* a_n, b_n *have quarter-power scales* E_a, E_b *and centre bounds* B_a, B_b*, their product receives the displayed scale and bound*

$$2(E_a B_b + B_a E_b), \qquad B_a B_b. \tag{3.31w-tr}$$

Addition, negation, scalar multiplication, multiplication, and semantic composition therefore construct the complete common trace without a caller-supplied continuity record. A reciprocal does require information which pointwise nonzeroness cannot constructively provide: displayed $\lambda > 0$ *with* $\lambda \leq \mathrm{N}(a_n)$ *at every sampled stage. Its channel then has scale* $E_a \lambda^{-2}$ *and bound* λ^{-1}*, by the exact difference-of-inverses identity. This gives automatic reciprocal and division traces from explicit base and output lower quadrances.*

Proof. Apply the squared triangle estimate to the exact product difference and use the two centre bounds, giving (3.31w-tr). Addition, negation, and scaling are special cases. For reciprocals use $a^{-1} - b^{-1} = (b-a)/(ab)$ and the displayed lower-quadrance certificates. Division is multiplication by the resulting inverse channel. □

Theorem 3.59 (analytic primitives preserve quadratic traces)**.** *Simultaneous finite Horner-depth and input refinement gives scale* $2(10 + 16E)$ *for the local exponential channel. The reduced complex-log chart* $2z(1 + z^2/3 + \cdots)$*—where the dots describe only the displayed finite recurrence—uses its proved finite input sensitivity and obtains scale* $2(5 + 64E)$ *on* $\mathrm{N}(z) \leq 1/9$*. Each becomes an outer quadratic trace and is then composed with the inner trace. Local sine and cosine are assembled from the two transported exponential traces and exact Gaussian-rational scaling. Thus arithmetic, composition, exponential, reduced-chart logarithm, sine, and cosine preserve the semantic certificate as productive CCL programs. The global range-reduced exponential and an anchored log correction remain the already proved bit-producing outer layers; their local differential certificate is the trace just described.*

Proof. Use the finite Horner tail and input-sensitivity bounds for exponential and the reduced logarithm, then insert the resulting outer certificate into (3.31x). Sine and cosine follow from their exponential definitions and the finite channel transport algebra. □

Proposition 3.60 (finite failure certificates)**.** *Partial constructors return a finite decision. Rejection names the missing positive denominator or output lower quadrance, input disk, safe chart, overlap, or branch-alignment certificate. It no longer asks the caller to invent a generic quadratic-transport proof for these automatic constructors.*

Proof. Each partial constructor compares its finite rational domain certificate. Success invokes the relevant transport theorem; failure returns the particular missing positive bound or chart field. □

Theorem 3.61 (productive finite trace programs)**.** *These nodes assemble into one finite program indexed by its complete semantic trace. A source node carries a quadratic trace; every other node computes its result index from the traces of its children. Composition is genuinely dependent: the outer program is indexed by the displayed inner displacement channel, not by an unrelated input sequence. The program therefore cannot attach arbitrary value, slope, and remainder names after the calculation. Its public request theorem simultaneously returns the quadratic law, extension and exact new length of all five live raw prefixes, exact ordinary-parser semantics, and a pairwise Hermitian-projective-quadrance cylinder bound below the requested rational target. Exact terminating certificates specialize literally, while the changing exponential-sample witness remains changing after nontrivial arithmetic and continues to emit every requested finite number of bits.*

Proof. Induct over the indexed program. Every constructor computes its result trace from its children, and dependent composition uses the actual inner displacement channel. Apply the persistent quadratic-trace theorem at the root. Exact inputs reduce to terminating traces; the changing witness is preserved by the same constructor equations. □

Definition 3.62 (finite Taylor and multivariable jets)**.** A Taylor jet is a low-to-high rational coefficient list, with finite truncation, Cauchy product, composition, and natural-number iteration of the formal derivative. A multivariable first jet is one Gaussian-rational value and one displayed finite list of partial coefficients; its dimension is that list's length.

Proposition 3.63 (finite jet table algebra)**.** *Taylor-jet truncation, Cauchy product, composition, and formal differentiation are finite list recursions. The product rule holds coefficient by coefficient. Directional coefficients of a multivariable jet are finite dot products and obey the addition, product, and chain rules.*

Proof. Induct over the displayed coefficient lists. Cauchy-product coefficients give the product rule, Horner recursion gives composition, and dot-product linearity gives the directional laws. Every recurrence terminates at a displayed list length. □

Theorem 3.64 (productive progressive coefficient tables)**.** *The progressive counterpart replaces each displayed coefficient by a coherent CCL channel. Addition, negation, coherent scaling, finite Cauchy product, finite Horner composition, truncation, and formal differentiation all recurse over those displayed lists. A single rational request is folded through the finitely many channel moduli. The table now retains the actual cursor belonging to every entry, rather than reconstructing a fresh cursor for each query. For every retained coefficient one joint request proves a radius and semantic allowance below the visible target, extension of the previous prefix by the exact requested bit count, and exact parser semantics. A finite schedule of differently sized requests preserves those origins and produces compatible names even when redundant-sector choices make the raw words textually different. Truncation transports precisely this contract for every retained entry; it does not leave an implicit obligation for an unmentioned tail. Exact rational coefficient tables specialize literally: after any positive joint request, their returned centres are the original displayed coefficient list.*

Proof. Fold one positive rational request through the finite list of channel moduli. Apply the persistent cursor theorem to every retained entry at the common state. Induction over the requested schedule preserves each cursor origin; truncation simply restricts the finite family. For exact tables every channel has allowance zero, so the returned centres are definitionally the input coefficients. □

Theorem 3.65 (scheduled compatibility of equivalent tables)**.** *Compatibility also holds when the two origins are merely operationally equivalent and their finite schedules are different. After each schedule, one common final request produces two extending prefixes of the exactly accounted lengths. Both retain exact parser semantics and each denotes a cylinder of pairwise Hermitian projective quadrance at most τ; the operational equivalence program additionally computes later rational sample indices whose separation is strictly below τ. The raw words need not coincide. This contract is applied to congruence of addition, subtraction, multiplication, and automatic exponential, and to the field laws, $\operatorname{Exp}(x+y) \simeq \operatorname{Exp}(x)\operatorname{Exp}(y)$, $\operatorname{Exp}(\operatorname{Log}(y;A)) \simeq y$, and the explicitly zero-aligned local reverse law. Hence these laws now govern actual emitted CCL prefixes rather than stopping at rational sample identities.*

Proof. After the two finite schedules, compute a common request using operational equivalence and the two table moduli. Each persistent cursor supplies its own extension, length, diameter, and parser certificate, while operational equality supplies later rational samples closer than the target. Apply this construction entrywise and use the already proved arithmetic, exponential, and logarithm congruence laws. □

Lemma 3.66 (finite coefficient evaluation budget)**.** *If a_j and $\widehat{a}_j$ are the actual and displayed entries with $\mathrm{N}(a_j - \widehat{a}_j) \le \eta$, and $\mathrm{N}(x) \le X$, a finite Horner recurrence computes a rational number $E(m,\eta,X)$ and proves*

$$\mathrm{N}\left(\sum_{j<m} a_j x^j - \sum_{j<m} \widehat{a}_j x^j\right) \le E(m,\eta,X). \tag{3.31w"}$$

The corresponding progressive theorem obtains the displayed centres from one actual table request. Appending a finite discarded coefficient list gives an exact algebraic retained-plus-tail identity and charges a separately displayed finite tail budget; there is no implicit infinite remainder. The same bound is exposed for finite Cauchy multiplication and Horner composition.

Proof. Induct through the finite Horner recurrence, applying the product and sum error bounds at each coefficient. The progressive statement takes all centres from one table request. Appending the explicitly discarded finite list proves the retained-plus-tail identity; the same induction handles Cauchy multiplication and composition. □

Theorem 3.67 (productive multivariable directional channels)**.** *The same construction handles a natural-number-sized multivariable first jet. Its partial channels form one finite table, and a directional channel is a finite dot-product recursion against a displayed Gaussian-rational list whose length must equal that table's displayed dimension. This directional output again has the complete finite CCL request contract. A second finite recurrence transports the displayed coefficient errors through the supplied direction list and proves the resulting rational directional-error budget. Thus "all coefficients" here always means membership in one explicit list, never an unbounded coordinate family.*

Proof. Check equality between the direction-list length and the displayed jet dimension, then recurse through the finite dot product. A second recursion propagates the coefficient-error allowances through the same direction list. Progressive arithmetic gives the complete CCL request contract for the resulting channel. □

Definition 3.68 (finite polynomial antiderivative and integral)**.** For $p(T) = \sum_{k=0}^{n} a_k T^k$, define

$$A_p(T) = c + \sum_{k=0}^{n} \frac{a_k}{k+1} T^{k+1}, \qquad I_p(a,b) = A_p(b) - A_p(a).$$

Theorem 3.69 (finite polynomial fundamental theorem)**.** *Coefficient arithmetic proves literally that $A_p' = p$. Applying the already constructed divided difference before diagonal substitution gives the infinitesimal-free fundamental theorem*

$$I_p(a,b) = (b-a)\Delta A_p(b,a), \qquad \Delta A_p(a,a) = p(a). \tag{3.31y}$$

This immediately constructs the exact finite initial-value solution of $y' = p(x)$ for displayed rational initial time and value. For $y' = y$, the degree-$(n+1)$ exponential polynomial has derivative equal to the degree-n polynomial, and its exact pointwise defect is the single last displayed Taylor monomial. These are deliberately finite ODE certificates; improving or subdividing them means requesting a larger natural degree, not completing an infinite trajectory.

Proof. Differentiate the displayed coefficient list to obtain $A_p' = p$. Apply the polynomial divided-difference identity before diagonal specialization, giving (3.31y). The ODE claims follow by the same finite coefficient calculation; the exponential defect is its single last displayed monomial. □

Definition 3.70 (finite path refinement)**.** At refinement n the caller returns one finite list of rational path cells. Cell j has displayed endpoints s_j, t_j, displacement $d_j = t_j - s_j$, a finite rational polynomial p_j, and a rational local squared-error allowance $\eta_j \geq 0$. Its displayed integral sample is

$$\widehat{I}_j = d_j \int_0^1 p_j(u)\, du, \qquad \widehat{I}_n = \sum_j \widehat{I}_j, \tag{3.31y'}$$

where every integral on the right is the coefficient calculation above. A finite observation with error e_j satisfies $N(e_j) \leq \eta_j$, so multiplicativity of Gaussian quadrance gives the explicit transported bound

$$N(d_j e_j) \leq N(d_j)\eta_j. \tag{3.31y"}$$

Lemma 3.71 (finite weighted path-error bound)**.** *Repeated use of $N(u+v) \leq 2(N(u)+N(v))$ computes a deliberately coarse rational allowance B_n by a fold over this particular finite cell list. Induction over that finite list, using (3.31y") at the head and the two-term squared-triangle estimate at each cons step, bounds the quadrance of the corresponding weighted finite error sum by B_n. There is no quantification over every point of a completed interval.*

Proof. Induct over the displayed cell list. At the head use (3.31y"); combine it with the induction hypothesis using the two-term squared triangle estimate. The resulting recurrence is the computable fold defining B_n. □

Definition 3.72 (certified integral refinement program)**.** A refinement program supplies a rational scale $\sigma \geq 0$ and checks, for each finite call,

$$B_n \leq \sigma 4^{-(n+1)}, \qquad N(\widehat{I}_m - \widehat{I}_n) \leq \sigma 4^{-(n+1)} \quad (n \leq m). \tag{3.31y"'}$$

Theorem 3.73 (a certified path integral is CCL-computable)**.** *The already proved finite search for the first geometric radius below a positive rational target is therefore a computable refinement modulus. It turns the samples $\widehat{I}_n$ into a coherent approximation and then into an ordinary persistent CCL cursor. For every displayed positive target and natural bit count, one finite request returns an enclosure below that target, a prefix of exactly that length, prefix persistence, the exact bit count, and exact parser semantics. Thus the integral is a CCL computation with a rational accuracy side channel, not a postulated completed value.*

Proof. The quarter-power scan computes a refinement index from the requested target. Equation (3.31y"') gives both later-sample and local-error bounds, so the samples form a coherent progressive enclosure process. Feed that process to the persistent CCL controller and request the displayed bit count at the same refined state. □

Proposition 3.74 (finite path-refinement algebra)**.** *All expected finite algebra is recorded at this operational boundary. Reversing a cell uses an explicit polynomial reparameterization certificate and negates its sample; a structural path reversal reverses both every cell and their finite list order. Adjacent path lists concatenate exactly; polynomial addition and rational scaling are linear. These operations now lift to refinement programs themselves: scaling computes the squared scale transport, addition computes the two-source geometric modulus, concatenation uses the literal appended path, and reversal reuses the original modulus after the proved sign identity. Where reversing order or reconciling two local partitions changes the deliberately coarse allowance fold, the caller supplies that finite rational inequality explicitly. Each resulting program therefore inherits the same productive CCL request theorem.*

Proof. Cell reversal is the displayed polynomial reparameterization and sign identity; list reversal and concatenation are structural recursions. Linearity holds coefficientwise. Compose the corresponding finite refinement moduli, using a supplied coarse-fold comparison when partitions change. □

Theorem 3.75 (refinement-program fundamental theorem)**.** *When the local models are exact, the refinement output is operationally equivalent to the terminating result of the coefficient integrator. Finally, a supplied finite antiderivative process records at every refinement both a literal coefficient derivative equality and the endpoint-difference equality. Summing those displayed differences gives the same sample at every index, hence an operationally equivalent productive CCL output. This is the finite fundamental theorem for the general refinement interface: antiderivatives are certified programs, not functions represented as infinite sets, and no infinitesimal is used.*

Proof. For an antiderivative process, sum the literal endpoint differences; intermediate endpoints cancel in the finite list, so every sample equals the coefficient integrator's endpoint difference. □

Definition 3.76 (zero-aligned logarithm selection data)**.** For a candidate logarithm output L over $\mathrm{Exp}(X)$, the selection data consist of: a rational point admitted by the source and target enclosures; positive denominator-quadrance and reduced-disk certificates; the source chart factor and correction; a displayed cell (c, ρ) with $0 < \rho \leq 1$ containing both X and L; and a finite local exponential-injectivity certificate for that pair in the cell.

Theorem 3.77 (selected logarithm–exponential inverse)**.** *Given zero-aligned selection data, the constructed chart over the actual CCL* $\mathrm{Exp}(X)$ *satisfies* $L - X \simeq 0$ *and*

$$\mathrm{Log}(\mathrm{Exp}(X); \textit{zero-aligned chart}) \simeq X. \tag{3.31z}$$

Merely knowing $\mathrm{Exp}(L - X) \simeq 1$ *is intentionally insufficient.*

Proof. The safe-overlap record constructs the anchored chart over the actual $\mathrm{Exp}(X)$ process. Its anchored inverse gives $\mathrm{Exp}(L) \simeq \mathrm{Exp}(X)$. Apply the supplied local injectivity certificate to the two displayed cell members, obtaining $L \simeq X$. Arithmetic congruence gives $L - X \simeq 0$, and additive cancellation yields (3.31z). □

Corollary 3.78 (productive finite calculus)**.** *Finite first jets, coefficient tables, multivariable directional jets, quadratic traces, and certified path integrals are closed under their stated finite operations. Exponential, guarded logarithm, sine, and cosine preserve the corresponding CCL contracts; the two finite fundamental theorems and the selected reverse logarithm law hold under their displayed certificates.*

Proof. Combine the focused results of this subsection. □

Formal verification:
`CCL.ProgressiveTraceProgram.Calculation.persistent_bitstream_and_quadratic_law`.
Formal verification:
`CCL.ProgressiveIntegration.RefinementProgram.AntiderivativeProcess.ftc_equivalent`.
Formal verification:
`CCL.LogBranchGeometry.SelectedLogarithm.logarithm_exponential_inverse`.

3.9 Fractional powers are finite root families

Complex qth root is a finite-valued operation, not a single-valued one. We separate its RFAP family, chart-selected member, exact rational candidates, and cost account.

Definition 3.79 (fractional-power polynomial)**.** For displayed $y \in \mathbb{Q}(i)$ and positive $q \in \mathbb{N}$, put

$$P_{y,q}(T) = T^q - y. \tag{3.32}$$

Theorem 3.80 (RFAP fractional-power family)**.** *The polynomial $P_{y,q}$ has degree exactly q, satisfies*

$$P_{y,q}(z) = z^q - y \tag{3.33}$$

for every displayed $z \in \mathbb{Q}(i)$, and its RFAP root family has multiplicity-weighted occurrence count exactly q.

Proof. The coefficient list is explicit, so its leading coefficient and degree are computed directly; finite Horner induction proves (3.33). The squarefree/multiplicity preprocessor followed by the RFAP box construction therefore returns a list of exact linear terminals and independently shrinking higher-degree CCL families whose multiplicity-weighted occurrence count is exactly q. For $y = 0$ this correctly permits the single distinct site 0 with multiplicity q; no artificial list of q unequal zeroes is manufactured. □

Definition 3.81 (chart-selected fractional power)**.** For anchored chart A, define

$$\mathsf{SelectedRoot}(Y; A, q) = \mathsf{ExpName}\big(q^{-1}\mathsf{LogName}(Y; A)\big), \tag{3.34}$$

Theorem 3.82 (selected root reconstruction)**.** *The selected root satisfies*

$$\mathsf{SelectedRoot}(Y; A, q)^q \simeq Y. \tag{3.35}$$

Proof. Finite induction applies the exponential addition law through the displayed q-fold CCL product. The rational exponent cancels, and the anchored exponential–logarithm inverse returns Y. □

The polynomial family, rather than $\exp(\log(y)/q)$, is the multi-valued operation $y^{1/q}$. The selected root is only one chart-selected member of (3.32). Likewise, the branch-selected rational power $Y^{p/q}$ is the displayed p-fold product of (3.34).

Definition 3.83 (sparse rational root candidate system)**.** Define the radius polynomial

$$R_{y,q}(S) = S^q - N(y). \tag{3.36}$$

For a displayed rational s, define the finite Lucas polynomials

$$A_0(X;s) = 2, \qquad A_1(X;s) = 2X, \qquad A_{n+2}(X;s) = 2XA_{n+1}(X;s) - sA_n(X;s). \tag{3.37}$$

For each surviving rational radius s, use the coordinate polynomials $A_q(X;s) - 2\Re(y)$ and $A_q(X;s) - 2\Re((-i)^q y)$.

Lemma 3.84 (root projection into the sparse system)**.** *If $z^q = y$ and $s = N(z)$, then $R_{y,q}(s) = 0$ and*

$$A_q(\Re z; N(z)) = 2\Re(z^q). \tag{3.38}$$

and

$$A_q(\Im z; s) = 2\Re((-i)^q y). \tag{3.39}$$

Proof. Multiplicativity gives $s^q = N(y)$ and hence the radius equation. Adding the nth powers of z and $\bar{z}$ proves (3.38) by finite induction through (3.37). Apply the same identity to $-iz$, whose real coordinate is $\Im z$, quadrance is s, and qth power is $(-i)^q y$. □

Each searched polynomial has $q+1$ dense coefficient slots and leading coefficient respectively $1, 2^q, 2^q$. Thus this path constructs no Sylvester matrix, determinant, scaled inverse, or resultant coefficient tower.

Lemma 3.85 (finite rational-root divisor pool)**.** *For any one of these displayed rational polynomials, trailing zero coefficients are removed, its finite canonical denominators are cleared, and its nonzero leading and constant integer coefficients are denoted L and C. If a nonzero root is in canonical reduced form a/b, homogeneous evaluation and* $\gcd(|a|, b) = 1$ *give directly*

$$|a| \mid |C|, \qquad b \mid |L|. \tag{3.40}$$

The executable pool tests zero separately and otherwise crosses the signed positive divisors of $|C|$ with the positive divisors of $|L|$. In particular it inspects no fraction merely because its numerator and denominator happen to lie below a common magnitude bound. If

$$D(P) = 1 + 2|C(P)|\,|L(P)|, \tag{3.41}$$

then the exact filtered candidate list for P has length at most $D(P)$.

Proof. Clear the finite canonical denominators and homogenize at the reduced fraction a/b. Coprimality makes a divide the constant endpoint and b divide the leading endpoint, proving (3.40). The pool has at most one zero and $2|C||L|$ signed divisor pairs, proving (3.41). □

Theorem 3.86 (complete automatic rational-root candidate search)**.** *Let $\mathcal{R}$ be the finite filtered candidate list for $R_{y,q}$, and write X_s, Y_s for the two coordinate polynomials at s. At most*

$$N(y,q) = \sum_{s \in \mathcal{R}} D(X_s)D(Y_s) \tag{3.42}$$

Gaussian candidates reach duplicate removal and the exact power pass. The sum here is a fold over the displayed finite list, not a sum over a named point collection. The final pass performs at most $N(y,q)$ power tests and the simple deduplicator at most $N(y,q)^2$ rational-pair comparisons. Every displayed Gaussian-rational solution of $z^q = y$ occurs in the promoted list and is returned as a terminating CCL name. If that distinct list has q entries, the degree-sharp check certifies it as complete.

Proof. For a supplied rational root z, the sparse-projection lemma puts its norm and coordinates into the three polynomial root lists. The divisor-pool lemma ensures that each reduced rational occurs in its finite pool, and exact Horner evaluation retains it. Thus the Cartesian product contains z, deduplication retains one copy, and the exact power test promotes it. The product of the coordinate bounds and the finite fold over $\mathcal{R}$ give (3.42); nested-loop counting gives the Horner, power, and deduplication bounds. A successful test supplies the terminating rational CCL certificate, and the degree-sharp conclusion follows by finite synthetic division. □

For example, the checked batch $[2,-2]$ for $y=4, q=2$ is returned without a caller-supplied candidate list. The search quantifies over no set of all Gaussian rationals and never tries to recognize an exact rational from an opaque non-terminating prefix.

Definition 3.87 (defensive bit-work ledger)**.** Put

$$\beta(n) = 1 + \lfloor \log_2 n \rfloor, \qquad \beta(a/b) = 1 + \beta(|a|) + \beta(b), \qquad \beta(u+iv) = \beta(u) + \beta(v), \tag{3.43}$$

with the displayed one-bit conventions at zero. In the defensive schoolbook model, a rational arithmetic operation with inputs of widths r_1, r_2 and normalized output width r_3 is charged

$$(r_1 + r_2 + r_3 + 1)^2 \quad \text{core bit work}, \qquad (r_3+1)^3 \quad \text{canonical-normalization work}. \tag{3.44}$$

The cubic term is a deliberately coarse allowance for the finite Euclidean normalization; it is kept separate rather than hidden inside a unit-cost field operation. The formal ledger follows the actual intermediate values of every Horner multiplication/addition and every repeated Gaussian power, the actual recursive denominator-clearance lists, every finite divisor scan, and the complete defensive duplicate-comparison pass. The sparse construction materializes exactly

$$(1 + 2|\mathcal{R}|)(q+1) \tag{3.45}$$

coefficient slots. Finally

$$B(y,q) = B_{\text{clear}} + B_{\text{sparse}} + B_{\text{divisors}} + B_{\text{normalize}} + B_{\text{Horner}} + B_{\text{dedup}} + B_{\text{power}} \tag{3.46}$$

Theorem 3.88 (defensive exact-root search bound)**.** *The complete automatic rational-root search performs at most $B(y,q)$ bit work in the schoolbook model (3.44), and materializes exactly the number of coefficient slots in (3.45).*

Proof. The ledger is a fully executable natural number computed from the displayed input and all finite intermediate lists. Summing the displayed charges for denominator clearance, sparse coefficient construction, divisor enumeration, Horner evaluation, duplicate comparisons, and exact power tests gives (3.46), together with the Horner, comparison, power-test, and final-output bounds. It is an upper account in the stated schoolbook model, not a claim about wall-clock time or an asymptotically optimal program. Faster integer multiplication and gcd, factored divisor generation, and a balanced finite deduplication table are now explicit places to sharpen (3.44)–(3.46). □

Corollary 3.89 (finite family of fractional powers)**.** *For displayed $y \in \mathbb{Q}(i)$ and positive q, $T^q - y$ has an RFAP family of multiplicity-weighted size q; every Gaussian-rational root is found and returned as a terminating CCL name within the candidate and bit-work bounds; and every anchored logarithm chart supplies one productive selected root whose qth power is operationally y.*

Proof. Combine the RFAP-family, selected-root, candidate-search, and work-ledger theorems. □

Formal verification:
`CCL.RootPowers.RootFamily.occurrenceCount_eq_degree`.

Formal verification:
`CCL.RootPowers.exactRoot_mem_automaticExactRoots`.

Formal verification:
`CCL.RootPowerBitComplexity.automaticExactRoots_bit_complexity`.

Formal verification:
`CCL.RootPowers.selectedRoot_power_reconstructs`.

4 Conclusion: arithmetic as finite requests

The canonical CCL of CCL-1 is left unchanged. Productive arithmetic instead uses overlapping safe output predicates and carries a rational accuracy sidechannel beside the ordinary raw prefix. This extra finite state is what allows an output to keep producing bits when an opaque computation approaches a canonical boundary or has an exact result that cannot be recognized from its input names alone.

Addition, subtraction, multiplication, and apartness-certified division are closed under this interface. Bounded range preparation and finite Horner recurrences extend it to exponential and logarithm, while explicit branch charts record exactly the information required for complex logarithms. Operational equivalence then expresses the field and transcendental laws at arbitrary requested accuracy without replacing the finite processes by completed values.

The same discipline supports divided differences, jets, Taylor bounds, coefficient-defined integration, progressive rational-path integration, trigonometric compositions, rational-function evaluation, and finite root families. Every conclusion is witnessed at a displayed finite request. Sharper complexity constants and broader automatically selected logarithm charts remain possible improvements, but they do not alter the finite computational foundation proved here.

A Independent machine verification

The references headed "Formal verification" point to the corresponding declarations in the companion Lean development. They are not premises of the paper proofs: each mathematical argument appears in the body above. The `CCL.Paper` and `CCL.Bridge.Paper` facades collect the combined CCL-1 and CCL-2 verification surface. A warning-fatal Lean build, the generated repository-wide `check-axioms.sh` audit, and an independent nanoda check verify that surface. The reported logical foundation is only `propext`; no declaration uses `Classical.choice`, `Quot.sound`, a completed scalar field, or a carrier-wide collection of values.

References

[1] N. J. Wildberger, *One dimensional metrical geometry*, arXiv:math/0701338, 2007.

[2] M. Beeler, R. W. Gosper, and R. Schroeppel, *HAKMEM*, MIT Artificial Intelligence Laboratory Memo AIM-239, 1972.

[3] J. M. Borwein, N. J. Calkin, S. B. Lindstrom, and A. Mattingly, Continued logarithms and associated continued fractions, *Experimental Mathematics* **26** (2017), no. 4, 412–429.

[4] J. M. Borwein, K. G. Hare, and J. G. Lynch, Generalized continued logarithms and related continued fractions, *Journal of Integer Sequences* **20** (2017), Article 17.5.7.

[5] M. J. Collins, Computing with continued logarithms, *arXiv:2602.07245*, 2026.

[6] M. Hockman, The geometry of Gaussian integer continued fractions, *Journal of Number Theory* **197** (2019), 145–167.

[7] A. Hurwitz, Über die Entwicklung complexer Grössen in Kettenbrüche, *Acta Mathematica* **11** (1887), 187–200.

[8] M. S. Kirsebom, Extreme value theory for Hurwitz complex continued fractions, *Journal of Theoretical Probability* **34** (2021), 2166–2197.

[9] G. González Robert, *Complex Continued Fractions*, Ph.D. thesis, Aarhus University, 2018.

[10] J. Shallit, Length of the continued logarithm algorithm on rational inputs, *Integers* **20** (2020), Paper A69.